02 File Integrity Monitoring System

02-file-integrity-monitor

Real-time SHA-256 file integrity monitoring with watchdog, an HMAC-signed SQLite baseline (external 0600 key) and a tamper-evident audit log.

Verified here: 38 passed; 0 failed; 0 skipped

Measured results

- 29/29 live file changes detected by the real watchdog Observer, median latency 302.4 ms
- Debounce: 50 rapid writes -> 202 raw OS events -> 1 alert
- 7/7 changes made while the watcher was off caught by offline scan
- SQLite baseline tampering (incl. rewriting a hash to hide an edit) detected via HMAC: 5/5 SQL attacks + wrong key + audit-log edits in tests
- 0 alerts through symlinks pointing outside the tree (2 skipped, never followed)
- 100 MiB re-hashed in 0.077 s (1303.6 MiB/s, page-cached)

Data provenance and caveats

- Generated only: scripts/make_sample.py builds 120 synthetic text files (seed 7) and 400 random 256 KiB blobs for the benchmark in a temp dir. No real system directories were scanned.
- All measurements are on generated files in a temp directory on Linux (inotify); no real system scan.
- Hashing throughput is with files in the OS page cache; cold-disk speed will be lower.
- Latency is dominated by the configured 300 ms debounce window.
- Threat model excludes an attacker running as the same user/root (can read the key); stated in README.
- Windows fallback path (no openat/O_NOFOLLOW, no 0600 check) is written but not tested here.

Project contents

README.md, requirements.txt, cli.py, app.py, src/__init__.py, src/fim.py, src/hashing.py, src/keystore.py, src/report.py, src/store.py, src/watcher.py, tests/conftest.py, tests/test_cli_report.py, tests/test_hashing.py, tests/test_keys_and_signing.py, tests/test_scan.py, tests/test_watcher_live.py, .gitignore, pytest.ini, samples/README.md, scripts/demo.py, scripts/make_sample.py

File Integrity Monitor (FIM)

Real-time file integrity monitoring in Python: SHA-256 baselines in SQLite, signed with HMAC-SHA256 using an external key, live change detection with `watchdog`, a tamper-evident audit log, and a Streamlit dashboard.

Problem

Attackers who get onto a server usually change files: they drop a web shell, add a cron job, edit an SSH or app config, or swap a binary. A File Integrity Monitor records a trusted **baseline** of every file's cryptographic hash and alerts when anything is **created, modified or deleted**. The hard part is not the hashing. It is making sure the attacker cannot quietly **edit the baseline itself** to hide the change, and being honest about what a monitor on the same machine can and cannot catch.

How it works / Architecture

```
            +-------------------------- monitored root --------------------------+
            |  regular files only, symlinks never followed, DB/key excluded      |
            +-----------------------------+-------------------------------------+
                                          |
       inotify / FSEvents / ReadDirectoryChangesW (watchdog Observer thread)
                                          v
  _Handler.on_any_event --> Debouncer (coalesce bursts, 0.3 s quiet window)
                                          v
  Watcher._process_batch:  1. verify baseline HMAC  --(invalid)--> TAMPER alert, stop trusting DB
                           2. lstat + safe open (openat + O_NOFOLLOW) + SHA-256
                           3. compare with last observed state --> created / modified / deleted
                                          v
  SQLite (fim.db)                          external key file (0600, NOT in the DB)
  +-------------------------+            +----------------------------------+
  | baseline(path, sha256,  |  HMAC-SHA256| 32 random bytes (hex)            |
  |   size, mode)           | <-----------| subkey("baseline") -> baseline_sig|
  | meta(root, generation,  |             | subkey("events")   -> event MACs |
  |   baseline_sig, ...)    |             +----------------------------------+
  | events(... prev_mac,mac)|  <- hash-chained audit log
  +-------------------------+
          |
          +--> cli.py report / export (CSV, JSON)   +--> app.py (Streamlit, opens DB read-only)
```

* **`init`** walks the tree, hashes every regular file, stores `(path, sha256, size, mode)` and signs the whole baseline with HMAC-SHA256.
* **`scan`** works **without the watcher**. It verifies the signature first, re-hashes the tree and reports drift. This catches anything that changed while the monitor was stopped or that the watcher missed.
* **`watch`** runs the watchdog Observer. Raw events are only hints: after the debounce window the watcher re-reads the file itself, so duplicated or reordered events cannot cause wrong alerts. On start it does a catch-up scan, and it re-verifies the baseline signature **before every batch** it processes.
* **`rebaseline`** is the only way to accept changes. It is a dry run unless you pass `--apply`, and it refuses to re-sign a baseline whose signature is already broken unless you pass `--force-untrusted`.
* **`verify`**, **`report`** and **`export`** check the signature and audit chain, summarise the events, and write CSV or JSON.

Security design decisions

Decision	Why
SHA-256 for file content	Collision- and second-preimage-resistant. MD5 and SHA-1 are not acceptable for integrity evidence.
HMAC-SHA256 over a **canonical** serialisation (sorted JSON of every baseline row plus root, generation, file count and key id)	Changing, adding or deleting **any** row, or pointing the DB at a different root, breaks the signature. Verification uses `hmac.compare_digest` (constant time).
Key kept in a separate file, never in the database	If the key sat next to the hashes, anyone who can write the DB could also recompute the signature. With the key separate, the classic bypass (edit a file, then update its stored hash) fails. The test `test_attacker_recomputing_hash_cannot_hide_modification` checks exactly this.
Key file created with `O_CREAT|O_EXCL|O_NOFOLLOW`, `fchmod 0600`, 256-bit random from `secrets`	The key is never silently overwritten, a planted symlink cannot redirect it, and the umask cannot widen its permissions. On load, a key readable by group or other, or owned by another user, is refused.
Domain-separated subkeys (`fim/baseline/v1`, `fim/events/v1`)	A MAC made for one purpose can never be replayed for the other.
HMAC-chained audit log with a MAC'd chain head	Editing, deleting or re-ordering events, including cutting off the newest ones, is detected.
No symlink traversal: `os.scandir(follow_symlinks=False)` plus component-by-component `openat(..., O_NOFOLLOW)`	A symlink to `/etc/shadow`, a symlink loop, or a directory swapped for a symlink between the walk and the open can never make the monitor read outside the root. FIFOs are opened `O_NONBLOCK` and skipped, so they cannot hang it.
Own DB, `-journal`/`-wal`/`-shm` files and key are excluded	They change all the time. Monitoring them would cause alert storms and feedback loops.
Safe defaults	`init` will not replace a baseline, `scan` never changes the baseline, `rebaseline` is a dry run, `export` will not overwrite files, and the dashboard opens SQLite read-only. Exit codes are `0` clean, `1` changes, `2` error, `3` **tamper**, so the tool fits into cron or CI.

Threat model. The monitor defends against an attacker who can modify monitored files and the SQLite database **but cannot read the key file**. Examples: a compromised web app running as a different user, a stolen DB backup, or a tampered copy. It does **not** defend against an attacker running as the same account as the monitor, or as root (see Limitations).

Project structure

```
02-file-integrity-monitor/
??? cli.py                # argparse CLI: init / scan / watch / rebaseline / verify / report / export
??? app.py                # Streamlit dashboard (tabs: Overview, Events, Integrity, Baseline)
??? src/
?   ??? hashing.py        # safe walk + SHA-256 (no symlinks, regular files only)
?   ??? keystore.py       # external 0600 key, subkeys, key fingerprint
?   ??? store.py          # SQLite schema, HMAC-signed baseline, HMAC-chained events
?   ??? fim.py            # init / scan / verify / rebaseline (safe defaults)
?   ??? watcher.py        # watchdog Observer + debouncer + live checks
?   ??? report.py         # text/JSON report, CSV/JSON export
??? scripts/
?   ??? make_sample.py    # synthetic file tree generator (no real system files)
?   ??? demo.py           # end-to-end demo + measurements -> outputs/
??? tests/                # 38 pytest tests (real watchdog events on temp dirs)
??? outputs/              # pytest_output.txt, sample_output.txt, demo_*.csv/json, charts/
```

??? requirements.txt pytest.ini .gitignore
```

## Setup & how to run

```bash
pip install -r requirements.txt

# 1. baseline (keep the key OUTSIDE the monitored tree; default ~/.fim/fim.key)
python cli.py --db fim.db --key ~/.fim/fim.key init ./some_dir

# 2. one-shot check (exit code 1 = changes, 3 = tampering)
python cli.py --db fim.db --key ~/.fim/fim.key scan

# 3. live monitoring (Ctrl-C to stop, or bound it)
python cli.py --db fim.db --key ~/.fim/fim.key watch --duration 60 --debounce 0.5

# 4. accept legitimate changes (dry run first, then apply)
python cli.py --db fim.db --key ~/.fim/fim.key rebaseline
python cli.py --db fim.db --key ~/.fim/fim.key rebaseline --apply

# 5. integrity check, report, export
python cli.py --db fim.db --key ~/.fim/fim.key verify
python cli.py --db fim.db --key ~/.fim/fim.key report --last 20
python cli.py --db fim.db --key ~/.fim/fim.key export --out events.csv

# demo on generated files + tests + dashboard
python scripts/demo.py
pytest -v
streamlit run app.py
```

Results (measured in this run, Linux/inotify, generated files only)

* **Tests:** 38 passed, 0 failed (`outputs/pytest_output.txt`): 6 hashing/symlink, 13 key
and HMAC tampering, 6 scan/rebaseline, **9 live watchdog** tests driving the real Observer
 on temp directories, and 4 CLI/report/export tests. The live suite was also re-run 5
times in a row with 9/9 passing each time, to check for flakiness.
* **Live detection:** **29/29 scripted changes detected (100.0%)**. These were 10 creates,
 10 content edits, 5 deletes, 3 permission changes and 1 write burst. Median latency was
302.4 ms, p95 303.1 ms, max 568.1 ms. The latency is dominated by the 300 ms debounce
window; the max is the burst, timed from its first write.
* **Debouncing:** a burst of 50 writes to one file produced **202 raw filesystem events
but 1 alert**.
* **Symlinks:** 2 symlinks pointing outside the tree were seen and skipped. Changes to
their targets produced **0 alerts**, so there was no traversal.
* **Offline detection (no watcher running):** 7/7 changes made while the monitor was
stopped were caught by `scan`.
* **Tampering:** rewriting a file's stored hash in SQLite to hide an edit was caught.
`scan` exited with code **3** (TAMPER ALERT). In the tests, 5 different SQL tampering
attacks, a wrong key, and edited or deleted audit events were all detected.
* **Hashing speed:** 400 files / 100.0 MiB re-scanned in 0.077 s (**1303.6 MiB/s**) on a
16-CPU box. The files were in the OS page cache, so cold-disk speed will be lower.
* **Demo audit log:** 42 HMAC-chained events (baseline=1, created=13, deleted=7,
modified=18, rebaseline=2, tamper=1).

Charts: `outputs/charts/detection_latency.png`, `events_by_type.png`,
`debounce_burst.png`.

Sample output (trimmed, real run; full text in `outputs/sample_output.txt`)
```

```
$ python cli.py --db ... --key ... scan
Scanned 125 files under /tmp/fim-demo/monitored in 0.002s (symlinks skipped: 1)
Baseline signature: VALID
7 change(s): created=2 modified=3 deleted=2
  MODIFIED  etc/nginx/file_0049.js [content]
  CREATED   opt/tool/bin/dropped_0.sh
  DELETED   opt/tool/bin/file_0012.sh
  ...
[exit code 1]

# attacker rewrites the stored hash of etc/nginx/file_0049.js in SQLite
$ python cli.py --db ... --key ... scan
TAMPER ALERT: baseline signature INVALID - database contents were altered
[exit code 3]
```

Limitations & future work

* **Same-account attacker:** anyone running as the monitor's user, or as root, can read the key, re-sign a forged baseline, kill the watcher, or delete the DB. HMAC protects the baseline against people who **cannot read the key**, not against the account that holds it. For real deployments: run the monitor as a dedicated user, keep the key on a separate host, in an HSM/TPM, or on read-only media, and ship alerts to a remote, append-only log or SIEM. A rollback to an older, validly signed copy of the whole DB is also not detected locally; a remote copy of the latest chain head would close that gap.
* **Watcher races:** watchdog events arrive after the fact. A file that is changed and then restored within the debounce window, or between two scans, produces no alert because the content is identical again. A file changed again while it is being hashed is caught on the next event or scan. The race between seeing an event and hashing the file is unavoidable for any user-space monitor.
* **Missed events:** inotify can drop events when its queue overflows, and it has a per-user watch limit (`fs.inotify.max_user_watches`). Anything that happens while the watcher is not running is invisible to it. That is why `watch` does a catch-up scan on start, and why running `scan` on a schedule (cron or a systemd timer) is recommended alongside it.
* **Scope:** only regular files are tracked (content plus permission bits). Owner/group, xattrs/ACLs, symlink targets and directories are not baselined. Signature verification per batch costs O(baseline size), which is fine for thousands of files; very large trees would want an incremental Merkle-tree signature.
* **Platforms:** tested on Linux. On Windows the race-free `openat` path and the 0600 key-permission check are not available; the code falls back to `lstat` plus open and skips the permission check.
* Future work: Merkle-tree signing, remote syslog/webhook alerts, ownership and xattr tracking, key rotation, and a systemd unit.

Resume bullets

* Built a real-time File Integrity Monitor in Python (watchdog, SQLite, SHA-256) that detected **29/29** live file changes with a **~302 ms** median latency, and collapsed a 50-write burst (**202** raw OS events) into **1** alert through debouncing.
* Designed a tamper-evident baseline signed with HMAC-SHA256 using an external 0600 key and domain-separated subkeys, plus an HMAC-chained audit log. Database edits that tried to hide a modified file were detected in every tested scenario (exit code 3).
* Hardened the scanner against symlink traversal and TOCTOU file swaps with `openat`/`O_NOFOLLOW` and fstat-verified regular files (0 alerts through symlinks pointing outside the tree), with safe defaults: dry-run re-baselining, no overwrites, and a read-only dashboard.
* Wrote 38 pytest tests, including 9 that drive the real watchdog Observer on temporary directories, plus a CLI (init/scan/watch/rebaseline/verify/report/export) and a Streamlit

dashboard; the scanner re-hashed 100 MiB in 0.077 s (page-cached).

```
# Demo on a GENERATED tree: 120 synthetic files (529500 bytes), seed 7

# 1. Create the signed baseline (key created with mode 0600 outside the tree)
$ python cli.py --db /tmp/fim-demo/state/fim.db --key /tmp/fim-demo/secrets/fim.key init /tmp/fim-demo/monitored
Baseline created for /tmp/fim-demo/monitored
  files hashed     : 120 (529500 bytes in 0.003s)
  symlinks skipped : 0 (never followed)
  database         : /tmp/fim-demo/state/fim.db
  HMAC key         : /tmp/fim-demo/secrets/fim.key (keep it outside the monitored tree and the DB host if possible)
[exit code 0]
$ stat -c '%a %n' /tmp/fim-demo/secrets/fim.key
600 /tmp/fim-demo/secrets/fim.key

# 2. Scan right away: nothing changed
$ python cli.py --db /tmp/fim-demo/state/fim.db --key /tmp/fim-demo/secrets/fim.key scan
Scanned 120 files under /tmp/fim-demo/monitored in 0.003s (symlinks skipped: 0)
Baseline signature: VALID
No changes against the baseline.
[exit code 0]

# 3. Live monitoring with watchdog (bounded to 4 s; changes made from another shell)
$ python cli.py --db ... --key ... watch --duration 4 --debounce 0.3
Watching /tmp/fim-demo/monitored (debounce 0.3s, for 4.0s) ...
[2026-10-27T08:18:40.156] CREATED   etc/app/injected.conf
[2026-10-27T08:18:40.158] MODIFIED  etc/app/file_0002.ini [content]
Stopped. raw filesystem events=8, processed batches=3, alerts=2, symlinks skipped=0

# Accept the CLI-demo changes explicitly before the measured run
$ python cli.py --db /tmp/fim-demo/state/fim.db --key /tmp/fim-demo/secrets/fim.key rebaseline --apply
1 change(s) between the baseline and the current tree:
  MODIFIED  etc/app/file_0002.ini [content]
New baseline signed (generation 2, 120 files).
[exit code 0]

# 4. Measured live run: 29/29 scripted changes detected, median latency 302.4 ms; burst of 50 writes -> 202 raw events -> 1 alert(s); symlink alerts: 0

# 5. Re-baseline is a DRY RUN by default
$ python cli.py --db /tmp/fim-demo/state/fim.db --key /tmp/fim-demo/secrets/fim.key rebaseline
29 change(s) between the baseline and the current tree:
  MODIFIED  etc/app/file_0002.ini [content]
  MODIFIED  etc/app/file_0005.txt [content]
  MODIFIED  etc/app/file_0006.ini [content]
  MODIFIED  etc/app/file_0008.css [content]
  MODIFIED  etc/app/file_0009.json [content]
  MODIFIED  etc/app/file_0016.sh [content]
  MODIFIED  etc/app/file_0018.js [content]
  ... (21 more lines)
  CREATED   var/config/new_9.conf
DRY RUN: nothing changed. Re-run with --apply to accept these changes as the new baseline.
[exit code 0]

#    ... and only re-signs with --apply
$ python cli.py --db /tmp/fim-demo/state/fim.db --key /tmp/fim-demo/secrets/fim.key rebaseline --apply
29 change(s) between the baseline and the current tree:
  MODIFIED  etc/app/file_0002.ini [content]
  MODIFIED  etc/app/file_0005.txt [content]
  MODIFIED  etc/app/file_0006.ini [content]
  ... (25 more lines)
  CREATED   var/config/new_9.conf
New baseline signed (generation 3, 125 files).
```

```
[exit code 0]

# 6. Changes made while the watcher was NOT running are caught by a scan
$ python cli.py --db /tmp/fim-demo/state/fim.db --key /tmp/fim-demo/secrets/fim.key scan
Scanned 125 files under /tmp/fim-demo/monitored in 0.002s (symlinks skipped: 1)
Baseline signature: VALID
7 change(s): created=2 modified=3 deleted=2
  MODIFIED  etc/nginx/file_0049.js [content]
  MODIFIED  etc/nginx/file_0051.ini [content]
  MODIFIED  etc/nginx/file_0060.ini [content]
  CREATED   opt/tool/bin/dropped_0.sh
  CREATED   opt/tool/bin/dropped_1.sh
  DELETED   opt/tool/bin/file_0012.sh
  DELETED   opt/tool/bin/file_0025.html
(recorded in the audit log; baseline NOT changed - use `rebaseline --apply` to accept)
[exit code 1]

# 7. Attack: rewrite the stored hash of etc/nginx/file_0049.js in SQLite to hide the edit

#    scan verifies the HMAC first
$ python cli.py --db /tmp/fim-demo/state/fim.db --key /tmp/fim-demo/secrets/fim.key scan
TAMPER ALERT: baseline signature INVALID - database contents were altered
[exit code 3]
$ python cli.py --db /tmp/fim-demo/state/fim.db --key /tmp/fim-demo/secrets/fim.key verify
TAMPER ALERT: baseline signature INVALID - database contents were altered
[exit code 3]

# 8. Report
$ python cli.py --db /tmp/fim-demo/state/fim.db --key /tmp/fim-demo/secrets/fim.key report --last
6
FILE INTEGRITY REPORT
============================================================
Monitored root     : /tmp/fim-demo/monitored
Baseline           : generation 3, 125 files, created 2026-10-27T08:18:45.844+00:00
Baseline signature : INVALID (baseline signature INVALID - database contents were altered)
Audit log chain    : VALID (42 events)
Event counts       : baseline=1, created=13, deleted=7, modified=18, rebaseline=2, tamper=1

Most recent 6 events:
  id  time (UTC)               type       source path / details
  42  2026-10-27T08:18:45.934  tamper     scan   - [baseline signature INVALID - database contents
 were altered]
  41  2026-10-27T08:18:45.894  deleted    scan   opt/tool/bin/file_0025.html
  40  2026-10-27T08:18:45.893  deleted    scan   opt/tool/bin/file_0012.sh
  39  2026-10-27T08:18:45.892  created    scan   opt/tool/bin/dropped_1.sh
  38  2026-10-27T08:18:45.892  created    scan   opt/tool/bin/dropped_0.sh
  37  2026-10-27T08:18:45.891  modified   scan   etc/nginx/file_0060.ini [content]
[exit code 3]

# 9. Hashing benchmark: 400 files / 100.0 MiB scanned in 0.077 s = 1303.6 MiB/s (files in the OS p
age cache; cold-disk speed will be lower)

```

02-file-integrity-monitor/requirements.txt

```
watchdog>=4.0
streamlit>=1.40
pandas>=2.0
plotly>=5.18
matplotlib>=3.8
pytest>=8.0
pytest-timeout>=2.2

```

```
#!/usr/bin/env python3
"""fim - File Integrity Monitor command line.

Exit codes: 0 = clean / ok, 1 = changes detected, 2 = usage or I/O error,
3 = TAMPERING detected (baseline signature or audit chain invalid).
"""
from __future__ import annotations

import argparse
import json
import os
import sys

from src.fim import FIM, BaselineExists, TamperError
from src.keystore import KeyError_
from src.report import build_report, export, format_report

DEFAULT_DB = "fim.db"
DEFAULT_KEY = os.path.join(os.path.expanduser("~"), ".fim", "fim.key")


def _fim(args) -> FIM:
    return FIM(args.db, args.key, root=getattr(args, "root", None),
               allow_insecure_key=args.allow_insecure_key)


def _print_changes(res) -> None:
    for ch in res.changes:
        extra = f" [{', '.join(ch.reasons)}]" if ch.reasons else ""
        print(f"  {ch.kind.upper():<9} {ch.path}{extra}")


def cmd_init(args) -> int:
    fim = _fim(args)
    res = fim.init(force=args.force)
    print(f"Baseline created for {fim.root}")
    print(f"  files hashed     : {res.stats.files} ({res.stats.bytes_hashed} bytes in {res.seconds
:.3f}s)")
    print(f"  symlinks skipped : {res.stats.skipped_symlinks} (never followed)")
    print(f"  database         : {fim.db_path}")
    print(f"  HMAC key         : {fim.key_path} (keep it outside the monitored tree and the DB hos
t if possible)")
    return 0


def cmd_scan(args) -> int:
    fim = _fim(args)
    try:
        res = fim.scan(record=not args.no_record)
    except TamperError as exc:
        print(f"TAMPER ALERT: {exc}", file=sys.stderr)
        return 3
    print(f"Scanned {res.stats.files} files under {fim.root} in {res.seconds:.3f}s "
          f"(symlinks skipped: {res.stats.skipped_symlinks})")
    print("Baseline signature: VALID")
    if res.clean:
        print("No changes against the baseline.")
        return 0
    print(f"{len(res.changes)} change(s): created={len(res.by_kind('created'))} "
          f"modified={len(res.by_kind('modified'))} deleted={len(res.by_kind('deleted'))}")
    _print_changes(res)
    if not args.no_record:
        print("(recorded in the audit log; baseline NOT changed - use `rebaseline --apply` to acce
pt)")
    return 1
```

```
63
64
65 def cmd_watch(args) -> int:
66     from src.watcher import watch
67     fim = _fim(args)
68
69     def show(ev):
70         det = ev.get("details", {})
71         extra = f" [{', '.join(det['reasons'])}]" if det.get("reasons") else ""
72         print(f"[{ev.get('ts', '')[:23]}] {ev['event_type'].upper():<9} {ev.get('path') or ''}{ext
   ra}", flush=True)
73
74     try:
75         print(f"Watching {fim.root} (debounce {args.debounce}s"
76               f"{', for %ss' % args.duration if args.duration else ', Ctrl-C to stop'}) ...", flus
   h=True)
77         w = watch(fim, duration=args.duration or None, debounce=args.debounce, on_event=show)
78     except TamperError as exc:
79         print(f"TAMPER ALERT: refusing to watch: {exc}", file=sys.stderr)
80         return 3
81     print(f"Stopped. raw filesystem events={w.raw_events}, processed batches={w.batches}, "
82           f"alerts={len(w.events)}, symlinks skipped={w.stats.skipped_symlinks}")
83     if w.tampered:
84         print(f"TAMPER ALERT: {w.tampered}", file=sys.stderr)
85         return 3
86     return 1 if w.events else 0
87
88
89 def cmd_rebaseline(args) -> int:
90     fim = _fim(args)
91     try:
92         res = fim.rebaseline(apply=args.apply, force_untrusted=args.force_untrusted)
93     except TamperError as exc:
94         print(f"TAMPER ALERT: {exc}", file=sys.stderr)
95         return 3
96     print(f"{len(res.changes)} change(s) between the baseline and the current tree:")
97     _print_changes(res)
98     if args.apply:
99         print(f"New baseline signed (generation {fim.store.get_meta('generation')}, {res.stats.fil
   es} files).")
100    else:
101        print("DRY RUN: nothing changed. Re-run with --apply to accept these changes as the new ba
   seline.")
102    return 0
103
104
105 def cmd_verify(args) -> int:
106    fim = _fim(args)
107    ok, problems = fim.verify()
108    if ok:
109        n = fim.store.get_meta("events_count", "0")
110        print(f"OK: baseline signature valid; audit chain of {n} events valid.")
111        return 0
112    for p in problems:
113        print(f"TAMPER ALERT: {p}", file=sys.stderr)
114    return 3
115
116
117 def cmd_report(args) -> int:
118    fim = _fim(args)
119    rep = build_report(fim, last=args.last)
120    print(json.dumps(rep, indent=2) if args.format == "json" else format_report(rep))
121    return 0 if rep["baseline"]["signature_valid"] and rep["audit_log"]["chain_valid"] else 3
122
123
```

```
def cmd_export(args) -> int:
    fim = _fim(args)
    n = export(fim, args.out, fmt=args.format, what=args.what, force=args.force)
    print(f"Exported {n} {args.what} rows to {args.out}")
    return 0


def build_parser() -> argparse.ArgumentParser:
    p = argparse.ArgumentParser(prog="fim", description="File Integrity Monitor (SHA-256 + HMAC-signed baseline)")
    p.add_argument("--db", default=DEFAULT_DB, help=f"SQLite database (default: {DEFAULT_DB})")
    p.add_argument("--key", default=DEFAULT_KEY, help=f"external HMAC key file, mode 0600 (default: {DEFAULT_KEY})")
    p.add_argument("--allow-insecure-key", action="store_true",
                   help="accept a key file readable by group/other (NOT recommended)")
    sub = p.add_subparsers(dest="command", required=True)

    s = sub.add_parser("init", help="hash a directory and create a signed baseline (creates the key if missing)")
    s.add_argument("root", help="directory to monitor")
    s.add_argument("--force", action="store_true", help="replace an existing baseline")
    s.set_defaults(func=cmd_init)

    s = sub.add_parser("scan", help="one-shot comparison against the signed baseline (no watcher needed)")
    s.add_argument("--no-record", action="store_true", help="pure dry run: do not write audit events")
    s.set_defaults(func=cmd_scan)

    s = sub.add_parser("watch", help="live monitoring with watchdog (verifies the baseline before each batch)")
    s.add_argument("--duration", type=float, default=0, help="stop after N seconds (default: run until Ctrl-C)")
    s.add_argument("--debounce", type=float, default=0.5, help="seconds of quiet before a path is processed")
    s.set_defaults(func=cmd_watch)

    s = sub.add_parser("rebaseline", help="accept the current tree as the new baseline (dry run unless --apply)")
    s.add_argument("--apply", action="store_true", help="actually re-sign the baseline")
    s.add_argument("--force-untrusted", action="store_true",
                   help="allow re-signing even if the current signature is invalid")
    s.set_defaults(func=cmd_rebaseline)

    s = sub.add_parser("verify", help="check baseline signature + audit-log chain only")
    s.set_defaults(func=cmd_verify)

    s = sub.add_parser("report", help="integrity status and recent events")
    s.add_argument("--format", choices=["text", "json"], default="text")
    s.add_argument("--last", type=int, default=20)
    s.set_defaults(func=cmd_report)

    s = sub.add_parser("export", help="export events or baseline to CSV/JSON")
    s.add_argument("--out", required=True)
    s.add_argument("--format", choices=["csv", "json"], default="csv")
    s.add_argument("--what", choices=["events", "baseline"], default="events")
    s.add_argument("--force", action="store_true", help="overwrite an existing output file")
    s.set_defaults(func=cmd_export)
    return p


def main(argv: list[str] | None = None) -> int:
    args = build_parser().parse_args(argv)
    try:
        return args.func(args)
```

02-file-integrity-monitor/cli.py (continued)

```
    except (KeyError_, BaselineExists, FileExistsError, NotADirectoryError, ValueError) as exc:
        print(f"error: {exc}", file=sys.stderr)
        return 2


if __name__ == "__main__":
    sys.exit(main())

```

```
"""Streamlit dashboard for the File Integrity Monitor.

    streamlit run app.py

Reads either the bundled demo export (outputs/demo_*.csv/json, works offline)
or a live FIM SQLite database (read-only) - plus, optionally, the key file to
verify the baseline signature and audit chain from the dashboard.
"""
from __future__ import annotations

import json
import os
import sqlite3

import pandas as pd
import plotly.express as px
import streamlit as st

HERE = os.path.dirname(os.path.abspath(__file__))
OUT = os.path.join(HERE, "outputs")

st.set_page_config(page_title="File Integrity Monitor", layout="wide")
st.title("File Integrity Monitor")
st.caption("SHA-256 baseline - HMAC-SHA256 signed - watchdog live events - HMAC-chained audit log")


@st.cache_data
def load_demo():
    ev = pd.read_csv(os.path.join(OUT, "demo_events.csv"))
    base = pd.read_csv(os.path.join(OUT, "demo_baseline.csv"))
    status = json.load(open(os.path.join(OUT, "demo_status.json")))
    metrics_path = os.path.join(OUT, "demo_metrics.json")
    metrics = json.load(open(metrics_path)) if os.path.exists(metrics_path) else {}
    return ev, base, status, metrics


def load_live(db_path: str, key_path: str | None):
    # Open the database read-only so the dashboard can never modify evidence.
    con = sqlite3.connect(f"file:{db_path}?mode=ro", uri=True)
    ev = pd.read_sql_query("SELECT * FROM events ORDER BY id", con)
    base = pd.read_sql_query("SELECT * FROM baseline ORDER BY path", con)
    meta = dict(con.execute("SELECT key, value FROM meta").fetchall())
    con.close()
    status = {"root": meta.get("root"),
              "baseline": {"generation": meta.get("generation"), "files": meta.get("file_count"),
                           "created_at": meta.get("created_at"), "signature_valid": None,
                           "signature_status": "not verified (no key given)"},
              "audit_log": {"events": len(ev), "chain_valid": None, "problems": []}}
    if key_path:
        from src.fim import FIM
        fim = FIM(db_path, key_path)
        ok_b, msg = fim.store.verify_baseline(fim.key)
        ok_e, n, problems = fim.store.verify_events(fim.key)
        fim.close()
        status["baseline"].update(signature_valid=ok_b, signature_status=msg)
        status["audit_log"].update(chain_valid=ok_e, problems=problems)
    return ev, base, status, {}


with st.sidebar:
    st.header("Data source")
    source = st.radio("Read from", ["Bundled demo export", "Live SQLite database"])
    db_path = key_path = None
    if source == "Live SQLite database":
```

```
        db_path = st.text_input("Database path", "fim.db")
        key_path = st.text_input("Key path (optional, to verify signatures)", "")
    st.markdown("The dashboard opens the database **read-only**.")

try:
    if source == "Live SQLite database" and db_path and os.path.exists(db_path):
        events, baseline, status, metrics = load_live(db_path, key_path or None)
    else:
        if source == "Live SQLite database":
            st.warning("Database not found - showing the bundled demo export instead.")
        events, baseline, status, metrics = load_demo()
except Exception as exc:  # show the problem instead of a stack trace
    st.error(f"Could not load data: {exc}")
    st.stop()

events["ts"] = pd.to_datetime(events["ts"], utc=True, format="ISO8601")
file_events = events[events["event_type"].isin(["created", "modified", "deleted"])]

tab_overview, tab_events, tab_integrity, tab_baseline = st.tabs(
    ["Overview", "Events", "Integrity", "Baseline"])

with tab_overview:
    b = status["baseline"]
    c1, c2, c3, c4, c5 = st.columns(5)
    c1.metric("Files in baseline", b.get("files"))
    c2.metric("Baseline generation", b.get("generation"))
    c3.metric("File change alerts", len(file_events))
    c4.metric("Tamper alerts", int((events["event_type"] == "tamper").sum()))
    sig = b.get("signature_valid")
    c5.metric("Baseline signature", "VALID" if sig else ("INVALID" if sig is False else "unverifie
d"))
    if sig is False:
        st.error(f"Tamper alert: {b.get('signature_status')}")
    counts = events["event_type"].value_counts().rename_axis("event_type").reset_index(name="count
")
    st.plotly_chart(px.bar(counts, x="event_type", y="count", color="event_type",
                           title="Audit events by type"), width="stretch")
    if len(file_events):
        st.plotly_chart(px.scatter(file_events, x="ts", y="event_type", color="source",
                                   hover_data=["path"], title="Timeline of file change alerts"),
                        width="stretch")
    m = metrics.get("metrics", {})
    if m:
        st.subheader("Demo measurements (generated files)")
        st.write(f"Live watchdog run: **{m['live_detected']}/{m['live_actions']}** changes detecte
d, "
                 f"median latency **{m['live_latency_ms_median']} ms** (debounce {m['debounce_s']}
 s). "
                 f"Burst of {m['burst_writes']} writes produced {m['burst_raw_fs_events']} raw fil
esystem "
                 f"events and **{m['burst_alerts']}** alert after debouncing.")
        lat = pd.DataFrame(metrics.get("live_actions", []))
        if not lat.empty:
            st.plotly_chart(px.box(lat, x="kind", y="latency_ms", points="all",
                                   title="Detection latency by change type (ms)"), width="stretch"
)

with tab_events:
    types = sorted(events["event_type"].unique())
    pick = st.multiselect("Event types", types, default=types)
    q = st.text_input("Path contains", "")
    view = events[events["event_type"].isin(pick)]
    if q:
        view = view[view["path"].fillna("").str.contains(q, regex=False)]
    st.dataframe(view[["id", "ts", "event_type", "source", "path", "old_sha256", "new_sha256", "de
```

```
tails"]],
                 width="stretch", hide_index=True)
    st.download_button("Download CSV", view.to_csv(index=False), "fim_events.csv", "text/csv")

with tab_integrity:
    b, a = status["baseline"], status["audit_log"]
    st.write(f"**Monitored root:** `{status.get('root')}`")
    st.write(f"**Baseline signature:** {b.get('signature_status')}")
    chain = a.get("chain_valid")
    st.write(f"**Audit log chain:** {'VALID' if chain else ('INVALID' if chain is False else 'unve
rified')} "
             f"({a.get('events')} events)")
    for p in a.get("problems", []):
        st.error(p)
    st.info("The HMAC key lives in a separate 0600 file, never in the database. Editing the databa
se "
            "(e.g. replacing a file's stored hash) breaks the signature unless the attacker can al
so read the key.")

with tab_baseline:
    st.write(f"{len(baseline)} files in the signed baseline")
    if "mode" in baseline:
        baseline = baseline.assign(mode_octal=baseline["mode"].apply(lambda m: oct(int(m))))
    st.dataframe(baseline, width="stretch", hide_index=True)

```

02-file-integrity-monitor/src/__init__.py

```
"""File Integrity Monitor (FIM): SHA-256 baselines, HMAC-signed, with live watchdog monitoring."""
__version__ = "1.0.0"

```

```
"""Core File Integrity Monitor: init, scan, verify, rebaseline.

Safe defaults:
* ``init`` refuses to replace an existing baseline (``force=True`` needed).
* ``scan`` only *reads* monitored files and never changes the baseline; it
  records drift as audit events (``record=False`` for a pure dry run).
* ``rebaseline`` is a dry run unless ``apply=True`` and refuses to re-sign a
  baseline whose signature does not verify unless ``force_untrusted=True``.
* Every operation that trusts the baseline verifies its HMAC first.
"""
from __future__ import annotations

import os
import time
from dataclasses import dataclass, field

from .hashing import FileRecord, WalkStats, snapshot
from .keystore import generate_key_file, load_key
from .store import Store


class TamperError(Exception):
    """The baseline or audit log failed HMAC verification."""


class BaselineExists(Exception):
    pass


@dataclass
class Change:
    kind: str                # created | modified | deleted
    path: str
    old: FileRecord | None
    new: FileRecord | None

    @property
    def reasons(self) -> list[str]:
        if self.kind != "modified" or not (self.old and self.new):
            return []
        r = []
        if self.old.sha256 != self.new.sha256:
            r.append("content")
        if self.old.mode != self.new.mode:
            r.append(f"mode {oct(self.old.mode)}->{oct(self.new.mode)}")
        return r


@dataclass
class ScanResult:
    changes: list[Change] = field(default_factory=list)
    stats: WalkStats = field(default_factory=WalkStats)
    seconds: float = 0.0

    def by_kind(self, kind: str) -> list[Change]:
        return [c for c in self.changes if c.kind == kind]

    @property
    def clean(self) -> bool:
        return not self.changes


def diff(baseline: dict[str, FileRecord], current: dict[str, FileRecord]) -> list[Change]:
    """Compare two snapshots (pure function, easy to test)."""
    changes: list[Change] = []
```

```
    for path in sorted(set(baseline) | set(current)):
        b, c = baseline.get(path), current.get(path)
        if b is None:
            changes.append(Change("created", path, None, c))
        elif c is None:
            changes.append(Change("deleted", path, b, None))
        elif (b.sha256, b.mode) != (c.sha256, c.mode):
            changes.append(Change("modified", path, b, c))
    return changes


def _norm(p: str) -> str:
    return os.path.normcase(os.path.abspath(p))


class FIM:
    def __init__(self, db_path: str, key_path: str, root: str | None = None,
                 allow_insecure_key: bool = False):
        self.db_path = os.path.abspath(db_path)
        self.key_path = os.path.abspath(key_path)
        self.store = Store(self.db_path)
        stored_root = self.store.get_meta("root")
        if root is None and stored_root is None:
            raise ValueError("no root given and the database has no baseline yet")
        # realpath() resolves a symlinked *root argument* once; nothing below
        # the root is ever followed.
        self.root = os.path.realpath(root) if root else stored_root
        self.allow_insecure_key = allow_insecure_key
        self._key: bytes | None = None
        # Our own state files must never be monitored (they change constantly).
        self.excluded = {_norm(self.db_path + s) for s in ("", "-journal", "-wal", "-shm")}
        self.excluded.add(_norm(self.key_path))

    # ------------------------------------------------------------ key access
    @property
    def key(self) -> bytes:
        if self._key is None:
            self._key = load_key(self.key_path, allow_insecure=self.allow_insecure_key)
        return self._key

    def close(self) -> None:
        self.store.close()

    # ------------------------------------------------------------ operations
    def init(self, force: bool = False) -> ScanResult:
        """Create the key (if missing) and a signed baseline of ``root``."""
        if not os.path.isdir(self.root):
            raise NotADirectoryError(self.root)
        if self.store.has_baseline() and not force:
            raise BaselineExists("a baseline already exists; use `rebaseline --apply` "
                                 "or `init --force` to replace it")
        if not os.path.exists(self.key_path):
            self._key = generate_key_file(self.key_path)
        res = ScanResult()
        t0 = time.perf_counter()
        current = snapshot(self.root, self.excluded, res.stats)
        res.seconds = time.perf_counter() - t0
        gen = int(self.store.get_meta("generation", "0")) + 1
        self.store.write_baseline(self.key, self.root, current.values(), generation=gen)
        self.store.append_event(self.key, "baseline", None, None, None, "admin",
                                {"files": len(current), "generation": gen,
                                 "skipped_symlinks": res.stats.skipped_symlinks})
        return res

    def verify(self) -> tuple[bool, list[str]]:
```

```
        """Check the baseline signature and the audit chain (no file access)."""
        problems: list[str] = []
        ok_b, msg = self.store.verify_baseline(self.key)
        if not ok_b:
            problems.append(msg)
        ok_e, _, ev_problems = self.store.verify_events(self.key)
        problems += ev_problems
        return (not problems), problems

    def require_trusted_baseline(self) -> dict[str, FileRecord]:
        """Verify the HMAC, then return the baseline. Raises TamperError."""
        ok, msg = self.store.verify_baseline(self.key)
        if not ok:
            raise TamperError(msg)
        return self.store.load_baseline()

    def record_tamper(self, reason: str, source: str) -> None:
        """Log a tamper alert (the chain itself may also be damaged; best effort)."""
        try:
            self.store.append_event(self.key, "tamper", None, None, None, source, {"reason": reaso
n})
        except Exception:  # pragma: no cover - never let alerting crash the caller
            pass

    def scan(self, record: bool = True) -> ScanResult:
        """Offline comparison of the filesystem against the signed baseline.

        Works without the watcher running, so changes made while the monitor
        was stopped (or that the watcher missed) are still caught.
        """
        try:
            baseline = self.require_trusted_baseline()
        except TamperError as exc:
            if record:
                self.record_tamper(str(exc), "scan")
            raise
        res = ScanResult()
        t0 = time.perf_counter()
        current = snapshot(self.root, self.excluded, res.stats)
        res.seconds = time.perf_counter() - t0
        res.changes = diff(baseline, current)
        if record:
            for ch in res.changes:
                self.store.append_event(
                    self.key, ch.kind, ch.path,
                    ch.old.sha256 if ch.old else None,
                    ch.new.sha256 if ch.new else None, "scan",
                    {"reasons": ch.reasons, "in_baseline": ch.old is not None})
        return res

    def rebaseline(self, apply: bool = False, force_untrusted: bool = False) -> ScanResult:
        """Show (dry run) or accept (apply=True) current state as the new baseline."""
        ok, msg = self.store.verify_baseline(self.key)
        if not ok and not force_untrusted:
            raise TamperError(msg + " - refusing to re-sign; investigate first or pass --force-unt
rusted")
        baseline = self.store.load_baseline()
        res = ScanResult()
        t0 = time.perf_counter()
        current = snapshot(self.root, self.excluded, res.stats)
        res.seconds = time.perf_counter() - t0
        res.changes = diff(baseline, current)
        if apply:
            gen = int(self.store.get_meta("generation", "0")) + 1
            self.store.write_baseline(self.key, self.root, current.values(), generation=gen)
```

02-file-integrity-monitor/src/fim.py (continued)

```
            self.store.append_event(self.key, "rebaseline", None, None, None, "admin", {
                "generation": gen, "files": len(current), "accepted_changes": len(res.changes),
                "previous_signature_valid": ok})
        return res

```

```
"""Safe file walking and hashing.

Design goals:
* Never follow symbolic links (neither while walking nor while opening a file).
* Only hash *regular* files (FIFOs, sockets and device nodes are skipped, so a
  named pipe dropped into the tree cannot block the monitor).
* Never escape the monitored root, even if a directory is swapped for a symlink
  between the walk and the open (POSIX: component-by-component ``openat`` with
  ``O_NOFOLLOW``).
"""
from __future__ import annotations

import errno
import hashlib
import os
import stat
from dataclasses import dataclass, field
from pathlib import PurePosixPath
from typing import Iterable, Iterator

CHUNK = 1024 * 1024  # 1 MiB read buffer

# POSIX systems get the race-free openat() path; Windows falls back to lstat+open.
_HAS_SAFE_OPEN = (
    hasattr(os, "O_NOFOLLOW")
    and hasattr(os, "O_DIRECTORY")
    and os.open in getattr(os, "supports_dir_fd", set())
)


class SymlinkRefused(Exception):
    """Raised when a path (or one of its parent directories) is a symlink."""


class NotRegularFile(Exception):
    """Raised when a path is not a regular file (FIFO, socket, device, dir)."""


@dataclass
class FileRecord:
    path: str      # path relative to the monitored root, POSIX separators
    sha256: str
    size: int
    mode: int      # permission bits (stat.S_IMODE)


@dataclass
class WalkStats:
    files: int = 0
    bytes_hashed: int = 0
    skipped_symlinks: int = 0
    skipped_special: int = 0
    skipped_excluded: int = 0
    errors: list[str] = field(default_factory=list)


def _check_rel(rel: str) -> tuple[str, ...]:
    parts = PurePosixPath(rel).parts
    if not parts or os.path.isabs(rel) or any(p in ("..", ".", "") for p in parts):
        raise ValueError(f"unsafe relative path: {rel!r}")
    return parts


def open_no_follow(root: str, rel: str) -> int:
    """Open ``root/rel`` read-only without following any symlink below ``root``.
```

```
66
67      Returns a raw file descriptor; the caller must close it.
68      """
69      parts = _check_rel(rel)
70      if not _HAS_SAFE_OPEN:  # pragma: no cover - Windows fallback
71          full = os.path.join(root, *parts)
72          if os.path.islink(full):
73              raise SymlinkRefused(rel)
74          return os.open(full, os.O_RDONLY | getattr(os, "O_BINARY", 0))
75
76      dfd = os.open(root, os.O_RDONLY | os.O_DIRECTORY)
77      try:
78          for comp in parts[:-1]:
79              try:
80                  nfd = os.open(comp, os.O_RDONLY | os.O_DIRECTORY | os.O_NOFOLLOW, dir_fd=dfd)
81              except OSError as exc:
82                  if exc.errno in (errno.ELOOP, errno.ENOTDIR):
83                      raise SymlinkRefused(rel) from exc
84                  raise
85              os.close(dfd)
86              dfd = nfd
87          try:
88              # O_NONBLOCK: opening a FIFO must never block the monitor.
89              return os.open(parts[-1], os.O_RDONLY | os.O_NOFOLLOW | os.O_NONBLOCK, dir_fd=dfd)
90          except OSError as exc:
91              if exc.errno == errno.ELOOP:
92                  raise SymlinkRefused(rel) from exc
93              raise
94      finally:
95          os.close(dfd)
96
97
98  def hash_file(root: str, rel: str) -> FileRecord:
99      """SHA-256 a regular file under ``root`` safely (no symlinks, no specials)."""
100     fd = open_no_follow(root, rel)
101     try:
102         st = os.fstat(fd)  # fstat on the *opened* fd: no TOCTOU on file type
103         if not stat.S_ISREG(st.st_mode):
104             raise NotRegularFile(rel)
105         h = hashlib.sha256()
106         size = 0
107         with os.fdopen(fd, "rb", closefd=False) as fh:
108             while True:
109                 chunk = fh.read(CHUNK)
110                 if not chunk:
111                     break
112                 h.update(chunk)
113                 size += len(chunk)
114         return FileRecord(rel, h.hexdigest(), size, stat.S_IMODE(st.st_mode))
115     finally:
116         os.close(fd)
117
118
119 def is_excluded(abs_path: str, excluded: Iterable[str]) -> bool:
120     """True if ``abs_path`` is one of the excluded files (DB, key, journals)."""
121     p = os.path.normcase(os.path.abspath(abs_path))
122     return p in excluded
123
124
125 def iter_files(root: str, excluded: set[str], stats: WalkStats, start_rel: str = "") -> Iterator[s
    tr]:
126     """Yield relative paths of regular files under ``root`` (or ``root/start_rel``).
127
128     Uses ``os.scandir`` with ``follow_symlinks=False`` everywhere: symlinked files
129     and symlinked directories are counted and skipped, never traversed.
```

```
    """
    stack = [start_rel]
    while stack:
        rel_dir = stack.pop()
        abs_dir = os.path.join(root, rel_dir) if rel_dir else root
        try:
            with os.scandir(abs_dir) as it:
                entries = sorted(it, key=lambda e: e.name)
        except OSError as exc:
            stats.errors.append(f"{rel_dir or '.'}: {exc.strerror}")
            continue
        for entry in entries:
            rel = f"{rel_dir}/{entry.name}" if rel_dir else entry.name
            if entry.is_symlink():
                stats.skipped_symlinks += 1
                continue
            if is_excluded(entry.path, excluded):
                stats.skipped_excluded += 1
                continue
            if entry.is_dir(follow_symlinks=False):
                stack.append(rel)
            elif entry.is_file(follow_symlinks=False):
                yield rel
            else:
                stats.skipped_special += 1


def snapshot(root: str, excluded: set[str], stats: WalkStats | None = None) -> dict[str, FileRecord]:
    """Hash every regular file under ``root``; returns ``{rel_path: FileRecord}``."""
    stats = stats if stats is not None else WalkStats()
    out: dict[str, FileRecord] = {}
    for rel in iter_files(root, excluded, stats):
        try:
            rec = hash_file(root, rel)
        except SymlinkRefused:
            stats.skipped_symlinks += 1
            continue
        except NotRegularFile:
            stats.skipped_special += 1
            continue
        except FileNotFoundError:
            continue  # vanished between listing and opening (race) - not an error
        except OSError as exc:
            stats.errors.append(f"{rel}: {exc.strerror}")
            continue
        out[rel] = rec
        stats.files += 1
        stats.bytes_hashed += rec.size
    return out

```

02-file-integrity-monitor/src/keystore.py

```
"""External HMAC key handling.

The HMAC key is deliberately stored OUTSIDE the SQLite database:
an attacker who can rewrite the database (to hide a modified file by updating
its stored hash) cannot produce a valid signature without also reading this
key. The key file is created with permission 0600 and loading refuses a key
that is readable by group/other or owned by another user.
"""
from __future__ import annotations

import hashlib
import hmac
import os
import secrets
import stat
import sys

KEY_BYTES = 32  # 256-bit key, matching HMAC-SHA256's block security


class KeyError_(Exception):
    """Problem with the key file (missing, malformed, unsafe permissions)."""


def generate_key_file(path: str, overwrite: bool = False) -> bytes:
    """Create a new random key at ``path`` with mode 0600 and return it.

    Uses O_CREAT|O_EXCL so an existing key is never silently replaced, and
    O_NOFOLLOW so a planted symlink cannot redirect the write.
    """
    parent = os.path.dirname(os.path.abspath(path))
    os.makedirs(parent, mode=0o700, exist_ok=True)
    if overwrite and os.path.lexists(path):
        os.unlink(path)
    flags = os.O_WRONLY | os.O_CREAT | os.O_EXCL | getattr(os, "O_NOFOLLOW", 0)
    try:
        fd = os.open(path, flags, 0o600)
    except FileExistsError as exc:
        raise KeyError_(f"key file already exists: {path} (refusing to overwrite)") from exc
    key = secrets.token_bytes(KEY_BYTES)
    try:
        if hasattr(os, "fchmod"):
            os.fchmod(fd, 0o600)  # enforce 0600 regardless of the process umask
        os.write(fd, key.hex().encode() + b"\n")
    finally:
        os.close(fd)
    return key


def load_key(path: str, allow_insecure: bool = False) -> bytes:
    """Load the key, refusing unsafe permissions unless ``allow_insecure``."""
    try:
        fd = os.open(path, os.O_RDONLY | getattr(os, "O_NOFOLLOW", 0))
    except FileNotFoundError as exc:
        raise KeyError_(f"key file not found: {path}") from exc
    except OSError as exc:
        raise KeyError_(f"cannot open key file {path}: {exc.strerror}") from exc
    try:
        st = os.fstat(fd)
        if not stat.S_ISREG(st.st_mode):
            raise KeyError_(f"key path is not a regular file: {path}")
        if sys.platform != "win32" and not allow_insecure:
            if st.st_mode & 0o077:
                raise KeyError_(
                    f"key file {path} has permissions {oct(stat.S_IMODE(st.st_mode))}; "
```

```
                    "it must be 0600 (chmod 600 it)"
                )
            if hasattr(os, "getuid") and st.st_uid != os.getuid():
                raise KeyError_(f"key file {path} is owned by another user")
        raw = os.read(fd, 4096).strip()
    finally:
        os.close(fd)
    try:
        key = bytes.fromhex(raw.decode())
    except ValueError as exc:
        raise KeyError_("key file is not valid hex") from exc
    if len(key) != KEY_BYTES:
        raise KeyError_(f"key must be {KEY_BYTES} bytes, got {len(key)}")
    return key


def subkey(master: bytes, purpose: str) -> bytes:
    """Derive a purpose-specific key (domain separation).

    The baseline signature and the audit-log chain use different subkeys so a
    MAC produced for one can never be replayed as the other.
    """
    return hmac.new(master, f"fim/{purpose}/v1".encode(), hashlib.sha256).digest()


def fingerprint(master: bytes) -> str:
    """Short non-secret key identifier stored in the DB to catch key mix-ups."""
    return hmac.new(master, b"fim/key-id/v1", hashlib.sha256).hexdigest()[:16]

```

```
"""Reporting and export helpers (text/JSON report, CSV/JSON export)."""
from __future__ import annotations

import csv
import json
import os
from typing import Any

from .fim import FIM


def build_report(fim: FIM, last: int = 20) -> dict[str, Any]:
    """Gather integrity status + event summary into one JSON-able dict."""
    ok_b, msg_b = fim.store.verify_baseline(fim.key)
    ok_e, n_events, ev_problems = fim.store.verify_events(fim.key)
    meta = fim.store.all_meta()
    return {
        "root": meta.get("root"),
        "baseline": {
            "generation": int(meta.get("generation", 0)),
            "created_at": meta.get("created_at"),
            "files": int(meta.get("file_count", 0)),
            "signature_valid": ok_b,
            "signature_status": msg_b,
        },
        "audit_log": {"events": n_events, "chain_valid": ok_e, "problems": ev_problems},
        "event_counts": fim.store.event_counts(),
        "recent_events": fim.store.list_events(limit=last),
    }


def format_report(rep: dict[str, Any]) -> str:
    b, a = rep["baseline"], rep["audit_log"]
    lines = [
        "FILE INTEGRITY REPORT",
        "=" * 60,
        f"Monitored root     : {rep['root']}",
        f"Baseline           : generation {b['generation']}, {b['files']} files, created {b['creat
ed_at']}",
        f"Baseline signature : {'VALID' if b['signature_valid'] else 'INVALID'} ({b['signature_sta
tus']})",
        f"Audit log chain    : {'VALID' if a['chain_valid'] else 'INVALID'} ({a['events']} events)
",
    ]
    for p in a["problems"]:
        lines.append(f"  ! {p}")
    counts = rep["event_counts"]
    lines.append("Event counts       : " + (", ".join(f"{k}={v}" for k, v in sorted(counts.items()
)) or "none"))
    lines.append("")
    lines.append(f"Most recent {len(rep['recent_events'])} events:")
    lines.append(f"{'id':>4}  {'time (UTC)':<23}  {'type':<10} {'source':<6} path / details")
    for e in rep["recent_events"]:
        det = e["details"]
        extra = ""
        if det.get("reasons"):
            extra = " [" + ", ".join(det["reasons"]) + "]"
        elif det.get("reason"):
            extra = " [" + det["reason"] + "]"
        elif e["path"] is None and det:
            extra = " " + json.dumps(det, sort_keys=True)
        lines.append(f"{e['id']:>4}  {e['ts'][:23]:<23}  {e['event_type']:<10} {e['source']:<6} {e
['path'] or '-'}{extra}")
    return "\n".join(lines)

```

```
61
62 def export(fim: FIM, out_path: str, fmt: str = "csv", what: str = "events", force: bool = False) -
   > int:
63     """Write events or baseline to CSV/JSON. Refuses to overwrite unless force."""
64     if os.path.exists(out_path) and not force:
65         raise FileExistsError(f"{out_path} exists (use --force to overwrite)")
66     if what == "events":
67         rows = list(reversed(fim.store.list_events()))
68         for r in rows:
69             r["details"] = json.dumps(r["details"], sort_keys=True)
70         cols = ["id", "ts", "event_type", "path", "old_sha256", "new_sha256", "source", "details"]
71     elif what == "baseline":
72         rows = [vars(r) for r in sorted(fim.store.load_baseline().values(), key=lambda r: r.path)]
73         cols = ["path", "sha256", "size", "mode"]
74     else:
75         raise ValueError("what must be 'events' or 'baseline'")
76     os.makedirs(os.path.dirname(os.path.abspath(out_path)), exist_ok=True)
77     tmp = out_path + ".tmp"
78     with open(tmp, "w", newline="", encoding="utf-8") as fh:
79         if fmt == "csv":
80             wr = csv.DictWriter(fh, fieldnames=cols)
81             wr.writeheader()
82             wr.writerows(rows)
83         elif fmt == "json":
84             json.dump(rows, fh, indent=2)
85         else:
86             raise ValueError("fmt must be csv or json")
87     os.replace(tmp, out_path)  # atomic
88     return len(rows)
89
```

```
"""SQLite storage: baseline, metadata and an HMAC-chained audit log.

Integrity model
---------------
* ``baseline`` rows plus the signed ``meta`` fields are serialised canonically
  (sorted JSON) and authenticated with HMAC-SHA256 (``baseline_sig``).
  Editing, adding or deleting ANY baseline row, or changing the root path or
  generation, invalidates the signature.
* Every audit event carries ``mac = HMAC(k_events, prev_mac || event fields)``
  forming a hash chain; the chain head (count + last mac) is itself MAC'd in
  ``meta`` so deleting events from the middle OR the end is detected.
"""
from __future__ import annotations

import hashlib
import hmac
import json
import os
import sqlite3
import threading
from datetime import datetime, timezone
from typing import Any, Iterable

from .hashing import FileRecord
from .keystore import fingerprint, subkey

SCHEMA_VERSION = 1
GENESIS = "0" * 64

SCHEMA = """
CREATE TABLE IF NOT EXISTS meta (
    key   TEXT PRIMARY KEY,
    value TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS baseline (
    path   TEXT PRIMARY KEY,
    sha256 TEXT NOT NULL,
    size   INTEGER NOT NULL,
    mode   INTEGER NOT NULL
);
CREATE TABLE IF NOT EXISTS events (
    id         INTEGER PRIMARY KEY AUTOINCREMENT,
    ts         TEXT NOT NULL,
    event_type TEXT NOT NULL,       -- created | modified | deleted | tamper | baseline | rebaseli
ne
    path       TEXT,
    old_sha256 TEXT,
    new_sha256 TEXT,
    source     TEXT NOT NULL,       -- watch | scan | admin
    details    TEXT NOT NULL,       -- JSON object
    prev_mac   TEXT NOT NULL,
    mac        TEXT NOT NULL
);
CREATE INDEX IF NOT EXISTS idx_events_ts ON events(ts);
"""

SIGNED_META = ("schema_version", "root", "created_at", "generation", "file_count", "key_id")


def utcnow() -> str:
    return datetime.now(timezone.utc).isoformat(timespec="milliseconds")


def _canon(obj: Any) -> bytes:
    return json.dumps(obj, sort_keys=True, separators=(",", ":"), ensure_ascii=False).encode("utf-
```

```
8")


class Store:
    """Thread-safe wrapper around one SQLite connection."""

    def __init__(self, db_path: str):
        self.db_path = os.path.abspath(db_path)
        new = not os.path.exists(self.db_path)
        os.makedirs(os.path.dirname(self.db_path), exist_ok=True)
        # isolation_level=None -> autocommit; we issue explicit BEGIN statements.
        self.conn = sqlite3.connect(self.db_path, timeout=10, isolation_level=None,
                                    check_same_thread=False)
        self.lock = threading.RLock()
        if new and hasattr(os, "chmod"):
            os.chmod(self.db_path, 0o600)
        self.conn.executescript(SCHEMA)

    def close(self) -> None:
        with self.lock:
            self.conn.close()

    # ------------------------------------------------------------------ meta
    def get_meta(self, key: str, default: str | None = None) -> str | None:
        row = self.conn.execute("SELECT value FROM meta WHERE key=?", (key,)).fetchone()
        return row[0] if row else default

    def all_meta(self) -> dict[str, str]:
        return dict(self.conn.execute("SELECT key, value FROM meta").fetchall())

    def _set_meta(self, key: str, value: Any) -> None:
        self.conn.execute("INSERT OR REPLACE INTO meta(key, value) VALUES (?, ?)", (key, str(value
)))

    def has_baseline(self) -> bool:
        return self.get_meta("baseline_sig") is not None

    # ------------------------------------------------------------- baseline
    def load_baseline(self) -> dict[str, FileRecord]:
        rows = self.conn.execute("SELECT path, sha256, size, mode FROM baseline").fetchall()
        return {p: FileRecord(p, h, s, m) for p, h, s, m in rows}

    def _baseline_payload(self) -> bytes:
        meta = self.all_meta()
        signed_meta = {k: meta.get(k) for k in SIGNED_META}
        rows = self.conn.execute(
            "SELECT path, sha256, size, mode FROM baseline ORDER BY path").fetchall()
        return _canon({"meta": signed_meta, "entries": [list(r) for r in rows]})

    def _sign(self, master: bytes) -> str:
        return hmac.new(subkey(master, "baseline"), self._baseline_payload(), hashlib.sha256).hexd
igest()

    def write_baseline(self, master: bytes, root: str, records: Iterable[FileRecord],
                       generation: int, created_at: str | None = None) -> str:
        """Replace the baseline atomically and sign it. Returns the signature."""
        records = list(records)
        with self.lock:
            self.conn.execute("BEGIN IMMEDIATE")
            try:
                self.conn.execute("DELETE FROM baseline")
                self.conn.executemany(
                    "INSERT INTO baseline(path, sha256, size, mode) VALUES (?,?,?,?)",
                    [(r.path, r.sha256, r.size, r.mode) for r in records])
                self._set_meta("schema_version", SCHEMA_VERSION)
```

```
                self._set_meta("root", root)
                self._set_meta("created_at", created_at or utcnow())
                self._set_meta("generation", generation)
                self._set_meta("file_count", len(records))
                self._set_meta("key_id", fingerprint(master))
                sig = self._sign(master)
                self._set_meta("baseline_sig", sig)
                self.conn.execute("COMMIT")
            except Exception:
                self.conn.execute("ROLLBACK")
                raise
        return sig

    def verify_baseline(self, master: bytes) -> tuple[bool, str]:
        """Recompute the HMAC over the stored baseline; constant-time compare."""
        with self.lock:
            stored = self.get_meta("baseline_sig")
            if stored is None:
                return False, "no baseline has been created (run init)"
            if self.get_meta("key_id") != fingerprint(master):
                return False, "key mismatch: this database was signed with a different key"
            expected = self._sign(master)
        if not hmac.compare_digest(stored, expected):
            return False, "baseline signature INVALID - database contents were altered"
        return True, "baseline signature valid"

    # ------------------------------------------------------------------ events
    def _event_mac(self, k: bytes, prev_mac: str, fields: list[Any]) -> str:
        return hmac.new(k, _canon([prev_mac] + fields), hashlib.sha256).hexdigest()

    def _head_mac(self, k: bytes, count: int, head: str) -> str:
        return hmac.new(k, _canon(["head", count, head]), hashlib.sha256).hexdigest()

    def append_event(self, master: bytes, event_type: str, path: str | None,
                     old_sha: str | None, new_sha: str | None, source: str,
                     details: dict | None = None, ts: str | None = None) -> dict:
        """Append one event to the HMAC chain (atomic read-modify-write)."""
        k = subkey(master, "events")
        ts = ts or utcnow()
        det = json.dumps(details or {}, sort_keys=True)
        with self.lock:
            self.conn.execute("BEGIN IMMEDIATE")
            try:
                count = int(self.get_meta("events_count", "0"))
                prev = self.get_meta("events_head", GENESIS)
                fields = [ts, event_type, path, old_sha, new_sha, source, det]
                mac = self._event_mac(k, prev, fields)
                cur = self.conn.execute(
                    "INSERT INTO events(ts, event_type, path, old_sha256, new_sha256, source, deta
ils, prev_mac, mac)"
                    " VALUES (?,?,?,?,?,?,?,?,?)", (*fields, prev, mac))
                self._set_meta("events_count", count + 1)
                self._set_meta("events_head", mac)
                self._set_meta("events_head_mac", self._head_mac(k, count + 1, mac))
                self.conn.execute("COMMIT")
            except Exception:
                self.conn.execute("ROLLBACK")
                raise
        return {"id": cur.lastrowid, "ts": ts, "event_type": event_type, "path": path,
                "old_sha256": old_sha, "new_sha256": new_sha, "source": source,
                "details": details or {}}

    def verify_events(self, master: bytes) -> tuple[bool, int, list[str]]:
        """Walk the audit chain. Returns (ok, events_checked, problems)."""
        k = subkey(master, "events")
```

```
        problems: list[str] = []
        with self.lock:
            rows = self.conn.execute(
                "SELECT id, ts, event_type, path, old_sha256, new_sha256, source, details, prev_mac, mac"
                " FROM events ORDER BY id").fetchall()
            count = int(self.get_meta("events_count", "0"))
            head = self.get_meta("events_head", GENESIS)
            head_mac = self.get_meta("events_head_mac")
        prev = GENESIS
        for row in rows:
            eid, *fields, prev_mac, mac = row
            if prev_mac != prev:
                problems.append(f"event {eid}: chain broken (an earlier event was removed or reordered)")
            if not hmac.compare_digest(mac, self._event_mac(k, prev_mac, list(fields))):
                problems.append(f"event {eid}: MAC invalid (event was edited)")
            prev = mac
        if count and (head_mac is None or not hmac.compare_digest(head_mac, self._head_mac(k, count, head))):
            problems.append("chain head MAC invalid (event counter/head was edited)")
        if len(rows) != count:
            problems.append(f"expected {count} events, found {len(rows)} (events deleted or injected)")
        elif rows and prev != head:
            problems.append("last event does not match the signed chain head (tail truncated)")
        return (not problems), len(rows), problems

    def list_events(self, limit: int | None = None, event_type: str | None = None) -> list[dict]:
        q = "SELECT id, ts, event_type, path, old_sha256, new_sha256, source, details FROM events"
        args: list[Any] = []
        if event_type:
            q += " WHERE event_type=?"
            args.append(event_type)
        q += " ORDER BY id DESC"
        if limit:
            q += " LIMIT ?"
            args.append(limit)
        with self.lock:
            rows = self.conn.execute(q, args).fetchall()
        cols = ["id", "ts", "event_type", "path", "old_sha256", "new_sha256", "source", "details"]
        out = [dict(zip(cols, r)) for r in rows]
        for e in out:
            e["details"] = json.loads(e["details"] or "{}")
        return out

    def event_counts(self) -> dict[str, int]:
        with self.lock:
            return dict(self.conn.execute(
                "SELECT event_type, COUNT(*) FROM events GROUP BY event_type").fetchall())

```

```
"""Real-time monitoring with watchdog + debouncing.

Flow::

    inotify/FSEvents/ReadDirectoryChangesW
            |  (watchdog Observer thread)
            v
    _Handler.on_any_event --> Debouncer.touch(path)      (coalesce bursts)
                                    |  after `delay` s of quiet
                                    v
                      Watcher._process_batch(paths)
                        1. verify baseline HMAC (refuse to trust a tampered DB)
                        2. lstat/hash each path safely (no symlinks)
                        3. compare with last observed state -> audit event

Watchdog events are *hints*, not facts: we always re-read the file system
state ourselves, which also makes us robust to duplicated or reordered events.
"""
from __future__ import annotations

import os
import stat
import threading
import time
from typing import Callable

from watchdog.events import FileSystemEventHandler
from watchdog.observers import Observer

from .fim import FIM, TamperError
from .hashing import NotRegularFile, SymlinkRefused, WalkStats, hash_file, is_excluded, iter_files

IGNORED_EVENT_TYPES = {"opened", "closed_no_write"}


class Debouncer:
    """Collect paths and release each one after ``delay`` seconds of quiet."""

    def __init__(self, delay: float, callback: Callable[[list[str]], None]):
        self.delay = delay
        self.callback = callback
        self._pending: dict[str, float] = {}
        self._cond = threading.Condition()
        self._stop = False
        self._thread = threading.Thread(target=self._run, name="fim-debouncer", daemon=True)
        self.raw_events = 0

    def start(self) -> None:
        self._thread.start()

    def touch(self, path: str) -> None:
        with self._cond:
            self.raw_events += 1
            self._pending[path] = time.monotonic() + self.delay
            self._cond.notify()

    def _take_due(self, force: bool = False) -> list[str]:
        now = time.monotonic()
        due = [p for p, t in self._pending.items() if force or t <= now]
        for p in due:
            del self._pending[p]
        return sorted(due)

    def _run(self) -> None:
        while True:
```

```
            with self._cond:
                if self._stop:
                    return
                if self._pending:
                    wait = max(0.0, min(self._pending.values()) - time.monotonic())
                    self._cond.wait(timeout=min(wait, 0.25) if wait > 0 else 0)
                else:
                    self._cond.wait(timeout=0.25)
                due = self._take_due()
            if due:
                self.callback(due)

    def stop(self, flush: bool = True) -> None:
        with self._cond:
            self._stop = True
            self._cond.notify()
        self._thread.join(timeout=5)
        if flush:
            with self._cond:
                due = self._take_due(force=True)
            if due:
                self.callback(due)


class _Handler(FileSystemEventHandler):
    def __init__(self, debouncer: Debouncer):
        super().__init__()
        self.debouncer = debouncer

    def on_any_event(self, event) -> None:  # noqa: D401 - watchdog callback
        if event.event_type in IGNORED_EVENT_TYPES:
            return
        paths = [event.src_path]
        if getattr(event, "dest_path", None):
            paths.append(event.dest_path)  # a move = delete(src) + create(dest)
        for p in paths:
            if isinstance(p, bytes):
                p = os.fsdecode(p)
            self.debouncer.touch(p)


class Watcher:
    """Live monitor. Use as a context manager or call start()/stop()."""

    def __init__(self, fim: FIM, debounce: float = 0.5, initial_scan: bool = True,
                 on_event: Callable[[dict], None] | None = None):
        self.fim = fim
        self.debounce = debounce
        self.initial_scan = initial_scan
        self.on_event = on_event
        self.stats = WalkStats()
        self.events: list[dict] = []
        self.tampered: str | None = None
        self.batches = 0
        self._observed: dict[str, tuple[str, int] | None] = {}
        self._lock = threading.Lock()
        self._debouncer = Debouncer(debounce, self._process_batch)
        self._observer = None

    # ------------------------------------------------------------ lifecycle
    def start(self) -> "Watcher":
        baseline = self.fim.require_trusted_baseline()  # raises TamperError
        self._observed = {p: (r.sha256, r.mode) for p, r in baseline.items()}
        self._debouncer.start()
        self._observer = Observer()
```

```
        self._observer.schedule(_Handler(self._debouncer), self.fim.root, recursive=True)
        self._observer.start()
        if self.initial_scan:
            # Catch anything that changed while we were not watching. Done after
            # the observer starts so nothing falls into a gap between the two.
            res = self.fim.scan(record=False)
            self._process_batch([os.path.join(self.fim.root, c.path) for c in res.changes])
        return self

    def stop(self) -> None:
        if self._observer is not None:
            self._observer.stop()
            self._observer.join(timeout=5)
        self._debouncer.stop(flush=True)

    def __enter__(self) -> "Watcher":
        return self.start()

    def __exit__(self, *exc) -> None:
        self.stop()

    @property
    def raw_events(self) -> int:
        return self._debouncer.raw_events

    # ------------------------------------------------------------ handling
    def _emit(self, kind: str, rel: str, old: tuple | None, new: tuple | None, details: dict) -> N
one:
        ev = self.fim.store.append_event(self.fim.key, kind, rel, old[0] if old else None,
                                         new[0] if new else None, "watch", details)
        self.events.append(ev)
        if self.on_event:
            self.on_event(ev)

    def _rel(self, abs_path: str) -> str | None:
        rel = os.path.relpath(os.path.abspath(abs_path), self.fim.root)
        if rel == "." or rel.startswith(".." + os.sep) or rel == "..":
            return None
        return rel.replace(os.sep, "/")

    def _process_batch(self, paths: list[str]) -> None:
        with self._lock:
            if self.tampered:
                return
            self.batches += 1
            # Verify the signed baseline BEFORE acting on changes: if someone
            # edited the DB we stop trusting it and raise a tamper alert.
            ok, msg = self.fim.store.verify_baseline(self.fim.key)
            if not ok:
                self.tampered = msg
                self.fim.record_tamper(msg, "watch")
                if self.on_event:
                    self.on_event({"event_type": "tamper", "path": None, "details": {"reason": msg
}})
                return
            baseline = self.fim.store.load_baseline()
            for p in paths:
                rel = self._rel(p)
                if rel is None or is_excluded(p, self.fim.excluded):
                    continue
                self._check(rel, baseline)

    def _check(self, rel: str, baseline) -> None:
        abs_path = os.path.join(self.fim.root, rel)
        try:
```

```
            st = os.lstat(abs_path)
        except FileNotFoundError:
            # Path (file or whole directory) is gone: report every observed file under it.
            gone = [rel] + [p for p in self._observed if p.startswith(rel + "/")]
            for p in gone:
                if self._observed.get(p) is not None and not os.path.lexists(os.path.join(self.fim
.root, p)):
                    self._emit("deleted", p, self._observed[p], None,
                               {"in_baseline": p in baseline})
                    self._observed[p] = None
            return
        if stat.S_ISLNK(st.st_mode):
            self.stats.skipped_symlinks += 1
            prev = self._observed.get(rel)
            if prev is not None:  # a monitored file was replaced by a symlink
                self._emit("modified", rel, prev, None,
                           {"reasons": ["replaced by symlink (not followed)"], "in_baseline": rel
in baseline})
                self._observed[rel] = None
            return
        if stat.S_ISDIR(st.st_mode):
            # New/moved directory: files may have been created before inotify
            # added a watch for it, so walk it ourselves (closes that race).
            for sub in iter_files(self.fim.root, self.fim.excluded, self.stats, start_rel=rel):
                self._check_file(sub, baseline)
            for p in [p for p in self._observed if p.startswith(rel + "/")]:
                if self._observed[p] is not None and not os.path.lexists(os.path.join(self.fim.roo
t, p)):
                    self._emit("deleted", p, self._observed[p], None, {"in_baseline": p in baselin
e})
                    self._observed[p] = None
            return
        if stat.S_ISREG(st.st_mode):
            self._check_file(rel, baseline)
        else:
            self.stats.skipped_special += 1

    def _check_file(self, rel: str, baseline) -> None:
        try:
            rec = hash_file(self.fim.root, rel)
        except (SymlinkRefused, NotRegularFile):
            self.stats.skipped_symlinks += 1
            return
        except FileNotFoundError:
            return self._check(rel, baseline)  # deleted mid-flight (race)
        except OSError as exc:
            self.stats.errors.append(f"{rel}: {exc.strerror}")
            return
        new = (rec.sha256, rec.mode)
        prev = self._observed.get(rel)
        base = baseline.get(rel)
        matches = base is not None and (base.sha256, base.mode) == new
        if prev is None:
            self._emit("created", rel, None, new, {"in_baseline": base is not None,
                                                   "matches_baseline": matches, "size": rec.size})
        elif prev != new:
            reasons = []
            if prev[0] != new[0]:
                reasons.append("content")
            if prev[1] != new[1]:
                reasons.append(f"mode {oct(prev[1])}->{oct(new[1])}")
            self._emit("modified", rel, prev, new, {"reasons": reasons, "in_baseline": base is not
 None,
                                                    "matches_baseline": matches, "size": rec.size}
)
```

```
        self._observed[rel] = new


def watch(fim: FIM, duration: float | None = None, debounce: float = 0.5,
          on_event: Callable[[dict], None] | None = None, poll: float = 0.2) -> Watcher:
    """Run the watcher for ``duration`` seconds (None = until KeyboardInterrupt)."""
    w = Watcher(fim, debounce=debounce, on_event=on_event)
    w.start()
    try:
        end = None if duration is None else time.monotonic() + duration
        while (end is None or time.monotonic() < end) and not w.tampered:
            time.sleep(poll)
    except KeyboardInterrupt:
        pass
    finally:
        w.stop()
    return w

```

02-file-integrity-monitor/tests/conftest.py

```
import os
import pytest

from src.fim import FIM


@pytest.fixture
def env(tmp_path):
    """A monitored tree, a DB inside it (must be excluded) and a key outside it."""
    root = tmp_path / "monitored"
    (root / "etc").mkdir(parents=True)
    (root / "bin").mkdir()
    (root / "etc" / "app.conf").write_text("port=8080\nmode=prod\n")
    (root / "etc" / "users.cfg").write_text("alice\nbob\n")
    (root / "bin" / "tool.sh").write_text("#!/bin/sh\necho hi\n")
    os.chmod(root / "bin" / "tool.sh", 0o755)
    keydir = tmp_path / "keys"
    db = root / "fim.db"            # deliberately inside the monitored root
    key = keydir / "fim.key"
    fim = FIM(str(db), str(key), root=str(root))
    yield {"root": root, "db": db, "key": key, "fim": fim, "tmp": tmp_path}
    fim.close()


def reopen(env):
    """Open a fresh FIM on the same DB/key (like a second CLI invocation)."""
    return FIM(str(env["db"]), str(env["key"]))

```

```python
import csv
import json
import os
import subprocess
import sys

import pytest

from src.report import build_report, export, format_report

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))


def run(*args):
    return subprocess.run([sys.executable, os.path.join(ROOT, "cli.py"), *args],
                          capture_output=True, text=True, timeout=60, cwd=ROOT)


def test_cli_exit_codes_end_to_end(tmp_path):
    root = tmp_path / "data"
    root.mkdir()
    (root / "f.txt").write_text("one\n")
    base = ["--db", str(tmp_path / "fim.db"), "--key", str(tmp_path / "k" / "fim.key")]
    assert run(*base, "init", str(root)).returncode == 0
    assert run(*base, "init", str(root)).returncode == 2          # refuses overwrite
    assert run(*base, "scan").returncode == 0
    (root / "f.txt").write_text("two\n")
    r = run(*base, "scan")
    assert r.returncode == 1 and "MODIFIED  f.txt" in r.stdout
    r = run(*base, "rebaseline")
    assert "DRY RUN" in r.stdout
    assert run(*base, "scan").returncode == 1                     # dry run accepted nothing
    assert run(*base, "rebaseline", "--apply").returncode == 0
    assert run(*base, "scan").returncode == 0
    assert run(*base, "verify").returncode == 0
    import sqlite3
    c = sqlite3.connect(tmp_path / "fim.db")
    c.execute("UPDATE baseline SET size=999")
    c.commit(); c.close()
    r = run(*base, "scan")
    assert r.returncode == 3 and "TAMPER ALERT" in r.stderr
    assert run(*base, "verify").returncode == 3


def test_cli_watch_bounded_duration(tmp_path):
    root = tmp_path / "data"
    root.mkdir()
    base = ["--db", str(tmp_path / "fim.db"), "--key", str(tmp_path / "fim.key")]
    assert run(*base, "init", str(root)).returncode == 0
    r = run(*base, "watch", "--duration", "1")
    assert r.returncode == 0 and "Stopped." in r.stdout


def test_cli_rejects_world_readable_key(tmp_path):
    root = tmp_path / "data"
    root.mkdir()
    key = tmp_path / "fim.key"
    base = ["--db", str(tmp_path / "fim.db"), "--key", str(key)]
    assert run(*base, "init", str(root)).returncode == 0
    os.chmod(key, 0o644)
    r = run(*base, "scan")
    assert r.returncode == 2 and "0600" in r.stderr


def test_report_and_export(env, tmp_path):
```

```
    fim, root = env["fim"], env["root"]
    fim.init()
    (root / "etc" / "app.conf").write_text("x\n")
    fim.scan()
    rep = build_report(fim)
    assert rep["baseline"]["signature_valid"] and rep["audit_log"]["chain_valid"]
    assert rep["event_counts"] == {"baseline": 1, "modified": 1}
    text = format_report(rep)
    assert "Baseline signature : VALID" in text and "etc/app.conf" in text
    out = tmp_path / "events.csv"
    assert export(fim, str(out)) == 2
    rows = list(csv.DictReader(open(out)))
    assert [r["event_type"] for r in rows] == ["baseline", "modified"]
    with pytest.raises(FileExistsError):
        export(fim, str(out))                                   # safe default
    jout = tmp_path / "baseline.json"
    export(fim, str(jout), fmt="json", what="baseline")
    assert {r["path"] for r in json.load(open(jout))} == {"etc/app.conf", "etc/users.cfg", "bin/to
ol.sh"}

```

```python
import hashlib
import os

import pytest

from src.hashing import SymlinkRefused, NotRegularFile, WalkStats, hash_file, snapshot


def test_sha256_matches_hashlib(tmp_path):
    data = os.urandom(3 * 1024 * 1024 + 17)   # spans several read chunks
    (tmp_path / "big.bin").write_bytes(data)
    rec = hash_file(str(tmp_path), "big.bin")
    assert rec.sha256 == hashlib.sha256(data).hexdigest()
    assert rec.size == len(data)


def test_hash_file_refuses_symlink(tmp_path):
    (tmp_path / "real.txt").write_text("x")
    os.symlink(tmp_path / "real.txt", tmp_path / "link.txt")
    with pytest.raises(SymlinkRefused):
        hash_file(str(tmp_path), "link.txt")


def test_hash_file_refuses_symlinked_parent_directory(tmp_path):
    outside = tmp_path / "outside"
    outside.mkdir()
    (outside / "secret.txt").write_text("do not read")
    root = tmp_path / "root"
    root.mkdir()
    os.symlink(outside, root / "dirlink")
    with pytest.raises(SymlinkRefused):
        hash_file(str(root), "dirlink/secret.txt")


def test_rejects_path_traversal(tmp_path):
    with pytest.raises(ValueError):
        hash_file(str(tmp_path), "../etc/passwd")


def test_fifo_is_skipped_not_blocking(tmp_path):
    os.mkfifo(tmp_path / "pipe")
    with pytest.raises(NotRegularFile):
        hash_file(str(tmp_path), "pipe")
    st = WalkStats()
    assert snapshot(str(tmp_path), set(), st) == {}
    assert st.skipped_special == 1


def test_walk_never_traverses_symlinks(tmp_path):
    outside = tmp_path / "outside"
    outside.mkdir()
    for i in range(5):
        (outside / f"s{i}.txt").write_text("secret")
    root = tmp_path / "root"
    (root / "d").mkdir(parents=True)
    (root / "d" / "ok.txt").write_text("ok")
    os.symlink(outside, root / "d" / "to_outside")           # dir symlink
    os.symlink(outside / "s0.txt", root / "file_link")        # file symlink
    os.symlink(root, root / "loop")                           # symlink loop
    st = WalkStats()
    snap = snapshot(str(root), set(), st)
    assert list(snap) == ["d/ok.txt"]
    assert st.skipped_symlinks == 3

```

```python
import os
import sqlite3
import stat

import pytest

from src.fim import FIM, BaselineExists, TamperError
from src.keystore import KeyError_, generate_key_file, load_key
from tests.conftest import reopen


def _sql(db, q, args=()):
    c = sqlite3.connect(db)
    c.execute(q, args)
    c.commit()
    c.close()


def test_key_file_created_0600_outside_db(env):
    env["fim"].init()
    mode = stat.S_IMODE(os.stat(env["key"]).st_mode)
    assert mode == 0o600
    key_hex = env["key"].read_text().strip()
    # the key never appears in the database file
    assert key_hex.encode() not in env["db"].read_bytes()
    assert bytes.fromhex(key_hex) not in env["db"].read_bytes()


def test_insecure_key_permissions_refused(tmp_path):
    k = tmp_path / "k.key"
    generate_key_file(str(k))
    os.chmod(k, 0o644)
    with pytest.raises(KeyError_, match="0600"):
        load_key(str(k))
    assert len(load_key(str(k), allow_insecure=True)) == 32


def test_key_never_overwritten(tmp_path):
    k = tmp_path / "k.key"
    generate_key_file(str(k))
    with pytest.raises(KeyError_):
        generate_key_file(str(k))


def test_init_baseline_excludes_db_and_key(env):
    res = env["fim"].init()
    base = env["fim"].store.load_baseline()
    assert set(base) == {"etc/app.conf", "etc/users.cfg", "bin/tool.sh"}
    assert res.stats.skipped_excluded >= 1        # fim.db sits inside the root
    ok, msg = env["fim"].store.verify_baseline(env["fim"].key)
    assert ok, msg


def test_init_refuses_to_replace_existing_baseline(env):
    env["fim"].init()
    with pytest.raises(BaselineExists):
        env["fim"].init()
    env["fim"].init(force=True)                  # explicit flag allowed
    assert env["fim"].store.get_meta("generation") == "2"


@pytest.mark.parametrize("attack", [
    "UPDATE baseline SET sha256='%s' WHERE path='etc/app.conf'" % ("ab" * 32),
    "DELETE FROM baseline WHERE path='etc/users.cfg'",
    "INSERT INTO baseline VALUES ('evil.sh', '%s', 1, 493)" % ("cd" * 32),
```

```
    "UPDATE baseline SET mode=511 WHERE path='bin/tool.sh'",
    "UPDATE meta SET value='/somewhere/else' WHERE key='root'",
])
def test_sqlite_tampering_detected_by_hmac(env, attack):
    env["fim"].init()
    env["fim"].close()
    _sql(env["db"], attack)
    fim = reopen(env)
    ok, msg = fim.store.verify_baseline(fim.key)
    assert not ok and "INVALID" in msg
    with pytest.raises(TamperError):
        fim.scan()
    assert fim.store.event_counts().get("tamper") == 1
    fim.close()


def test_attacker_recomputing_hash_cannot_hide_modification(env):
    """Classic FIM bypass: modify a file AND update its hash in the DB."""
    import hashlib
    env["fim"].init()
    env["fim"].close()
    new = b"port=8080\nmode=prod\nbackdoor=1\n"
    (env["root"] / "etc" / "app.conf").write_bytes(new)
    _sql(env["db"], "UPDATE baseline SET sha256=?, size=? WHERE path='etc/app.conf'",
         (hashlib.sha256(new).hexdigest(), len(new)))
    fim = reopen(env)
    with pytest.raises(TamperError):
        fim.scan()
    fim.close()


def test_wrong_key_rejected(env, tmp_path):
    env["fim"].init()
    other = tmp_path / "other.key"
    generate_key_file(str(other))
    fim = FIM(str(env["db"]), str(other))
    ok, msg = fim.store.verify_baseline(fim.key)
    assert not ok and "different key" in msg
    fim.close()


def test_audit_chain_detects_deleted_and_edited_events(env):
    fim = env["fim"]
    fim.init()
    (env["root"] / "etc" / "app.conf").write_text("changed\n")
    (env["root"] / "new.txt").write_text("new\n")
    fim.scan()                                  # 2 more events
    ok, n, _ = fim.store.verify_events(fim.key)
    assert ok and n == 3
    fim.close()
    _sql(env["db"], "UPDATE events SET path='innocent.txt' WHERE id=2")
    f2 = reopen(env)
    ok, _, problems = f2.store.verify_events(f2.key)
    assert not ok and any("edited" in p for p in problems)
    f2.close()
    _sql(env["db"], "DELETE FROM events WHERE id=3")   # remove the tail event
    f3 = reopen(env)
    ok, _, problems = f3.store.verify_events(f3.key)
    assert not ok and any("deleted" in p or "truncated" in p for p in problems)
    f3.close()
```

```python
import os
import sqlite3

import pytest

from src.fim import FIM, TamperError, diff
from src.hashing import FileRecord
from tests.conftest import reopen


def test_diff_pure_function():
    a = {"x": FileRecord("x", "1", 1, 0o644), "y": FileRecord("y", "2", 1, 0o644)}
    b = {"x": FileRecord("x", "1", 1, 0o755), "z": FileRecord("z", "3", 1, 0o644)}
    kinds = {(c.kind, c.path) for c in diff(a, b)}
    assert kinds == {("modified", "x"), ("deleted", "y"), ("created", "z")}


def test_scan_detects_create_modify_delete_and_mode(env):
    fim, root = env["fim"], env["root"]
    fim.init()
    assert fim.scan().clean
    (root / "etc" / "app.conf").write_text("port=22\n")     # modify content
    os.remove(root / "etc" / "users.cfg")                     # delete
    (root / "etc" / "cron.d").mkdir()
    (root / "etc" / "cron.d" / "job").write_text("* * * * * x\n")  # create (nested)
    os.chmod(root / "bin" / "tool.sh", 0o777)                 # permission change
    res = fim.scan()
    got = {(c.kind, c.path) for c in res.changes}
    assert got == {("modified", "etc/app.conf"), ("deleted", "etc/users.cfg"),
                   ("created", "etc/cron.d/job"), ("modified", "bin/tool.sh")}
    tool = [c for c in res.changes if c.path == "bin/tool.sh"][0]
    assert tool.reasons == ["mode 0o755->0o777"]


def test_scan_is_read_only_for_baseline(env):
    fim, root = env["fim"], env["root"]
    fim.init()
    sig = fim.store.get_meta("baseline_sig")
    (root / "etc" / "app.conf").write_text("changed\n")
    fim.scan()
    fim.scan(record=False)
    assert fim.store.get_meta("baseline_sig") == sig      # baseline untouched
    assert fim.store.event_counts()["modified"] == 1      # record=False wrote nothing
    assert len(fim.scan().changes) == 1                   # drift persists until accepted


def test_rebaseline_dry_run_then_apply(env):
    fim, root = env["fim"], env["root"]
    fim.init()
    (root / "new.txt").write_text("approved change\n")
    dry = fim.rebaseline()
    assert [c.path for c in dry.changes] == ["new.txt"]
    assert "new.txt" not in fim.store.load_baseline()     # dry run changed nothing
    fim.rebaseline(apply=True)
    assert "new.txt" in fim.store.load_baseline()
    assert fim.store.get_meta("generation") == "2"
    assert fim.scan().clean
    assert fim.verify() == (True, [])


def test_rebaseline_refuses_tampered_baseline(env):
    env["fim"].init()
    env["fim"].close()
    c = sqlite3.connect(env["db"])
    c.execute("DELETE FROM baseline WHERE path='etc/app.conf'")
```

```
    c.commit(); c.close()
    fim = reopen(env)
    with pytest.raises(TamperError):
        fim.rebaseline(apply=True)
    fim.rebaseline(apply=True, force_untrusted=True)      # explicit override only
    assert fim.store.verify_baseline(fim.key)[0]
    fim.close()


def test_symlink_created_in_tree_is_not_followed_by_scan(env, tmp_path):
    fim, root = env["fim"], env["root"]
    fim.init()
    outside = tmp_path / "outside"
    outside.mkdir()
    (outside / "passwd").write_text("root:x:0:0\n")
    os.symlink(outside, root / "evil_dir")
    os.symlink(outside / "passwd", root / "etc" / "evil_file")
    res = fim.scan()
    assert res.clean                                      # nothing hashed through links
    assert res.stats.skipped_symlinks == 2

```

02-file-integrity-monitor/tests/test_watcher_live.py

```
"""These tests drive the REAL watchdog Observer (inotify on Linux) on a temp dir."""
import os
import sqlite3
import time

import pytest

from src.fim import TamperError
from src.watcher import Watcher

pytestmark = pytest.mark.live
DEBOUNCE = 0.2


def wait_for(pred, timeout=10.0, step=0.05):
    end = time.monotonic() + timeout
    while time.monotonic() < end:
        if pred():
            return True
        time.sleep(step)
    return pred()


def kinds(w):
    return [(e["event_type"], e["path"]) for e in w.events]


def test_live_create_modify_delete(env):
    fim, root = env["fim"], env["root"]
    fim.init()
    with Watcher(fim, debounce=DEBOUNCE) as w:
        (root / "etc" / "new.conf").write_text("hello\n")
        assert wait_for(lambda: ("created", "etc/new.conf") in kinds(w))
        (root / "etc" / "app.conf").write_text("port=1\n")
        assert wait_for(lambda: ("modified", "etc/app.conf") in kinds(w))
        os.remove(root / "etc" / "users.cfg")
        assert wait_for(lambda: ("deleted", "etc/users.cfg") in kinds(w))
    # every live alert is in the HMAC-chained audit log
    db_events = {(e["event_type"], e["path"], e["source"]) for e in fim.store.list_events()}
    assert ("created", "etc/new.conf", "watch") in db_events
    assert ("modified", "etc/app.conf", "watch") in db_events
    assert ("deleted", "etc/users.cfg", "watch") in db_events
    assert fim.store.verify_events(fim.key)[0]


def test_live_debounce_coalesces_burst(env):
    fim, root = env["fim"], env["root"]
    fim.init()
    target = root / "etc" / "app.conf"
    with Watcher(fim, debounce=0.4) as w:
        for i in range(25):                          # 25 rapid writes
            target.write_text(f"v{i}\n")
            time.sleep(0.005)
        assert wait_for(lambda: ("modified", "etc/app.conf") in kinds(w))
        time.sleep(0.6)
    assert w.raw_events >= 25                         # many raw inotify events ...
    assert kinds(w).count(("modified", "etc/app.conf")) == 1   # ... one alert


def test_live_rename_is_delete_plus_create(env):
    fim, root = env["fim"], env["root"]
    fim.init()
    with Watcher(fim, debounce=DEBOUNCE) as w:
        os.rename(root / "etc" / "users.cfg", root / "etc" / "users.cfg.bak")
        assert wait_for(lambda: {("deleted", "etc/users.cfg"),
```

```
                                ("created", "etc/users.cfg.bak")} <= set(kinds(w)))


def test_live_new_directory_with_files(env):
    fim, root = env["fim"], env["root"]
    fim.init()
    with Watcher(fim, debounce=DEBOUNCE) as w:
        d = root / "dropper"
        d.mkdir()
        for i in range(3):
            (d / f"payload{i}.bin").write_bytes(os.urandom(64))
        assert wait_for(lambda: sum(1 for k, p in kinds(w) if k == "created" and p.startswith("dro
pper/")) == 3)


def test_live_symlinks_not_traversed(env, tmp_path):
    fim, root = env["fim"], env["root"]
    fim.init()
    outside = tmp_path / "outside"
    outside.mkdir()
    (outside / "secret.txt").write_text("secret\n")
    with Watcher(fim, debounce=DEBOUNCE) as w:
        os.symlink(outside, root / "linkdir")
        os.symlink(outside / "secret.txt", root / "etc" / "linkfile")
        (root / "marker.txt").write_text("m\n")       # a real change to sync on
        assert wait_for(lambda: ("created", "marker.txt") in kinds(w))
        (outside / "secret.txt").write_text("changed outside\n")
        (outside / "another.txt").write_text("x\n")
        time.sleep(0.6)
    paths = [p for _, p in kinds(w)]
    assert not any(p and (p.startswith("linkdir") or p == "etc/linkfile") for p in paths)
    assert w.stats.skipped_symlinks >= 2


def test_live_db_and_key_activity_is_ignored(env):
    fim, root = env["fim"], env["root"]
    fim.init()
    with Watcher(fim, debounce=DEBOUNCE) as w:
        (root / "a.txt").write_text("a\n")            # generates DB writes inside root
        assert wait_for(lambda: ("created", "a.txt") in kinds(w))
        time.sleep(0.5)
    assert not any(p and p.startswith("fim.db") for _, p in kinds(w))


def test_live_watcher_refuses_tampered_baseline(env):
    env["fim"].init()
    c = sqlite3.connect(env["db"])
    c.execute("UPDATE baseline SET sha256=? WHERE path='etc/app.conf'", ("ef" * 32,))
    c.commit(); c.close()
    with pytest.raises(TamperError):
        Watcher(env["fim"], debounce=DEBOUNCE).start()


def test_live_tamper_while_running_stops_trusting_db(env):
    fim, root = env["fim"], env["root"]
    fim.init()
    with Watcher(fim, debounce=DEBOUNCE) as w:
        c = sqlite3.connect(env["db"])
        c.execute("DELETE FROM baseline WHERE path='etc/app.conf'")   # attacker edits DB
        c.commit(); c.close()
        (root / "etc" / "app.conf").write_text("evil\n")              # then the file
        assert wait_for(lambda: w.tampered is not None)
    assert "INVALID" in w.tampered
    assert fim.store.event_counts().get("tamper") == 1

```

```
130
131 def test_initial_scan_catches_offline_changes(env):
132     fim, root = env["fim"], env["root"]
133     fim.init()
134     (root / "etc" / "app.conf").write_text("changed while monitor was down\n")
135     with Watcher(fim, debounce=DEBOUNCE) as w:
136         assert wait_for(lambda: ("modified", "etc/app.conf") in kinds(w))
137
```

02-file-integrity-monitor/.gitignore

```
__pycache__/
*.pyc
.pytest_cache/
.venv/
venv/
*.db
*.db-journal
*.db-wal
*.db-shm
*.key
fim.key
.fim/

```

02-file-integrity-monitor/pytest.ini

```
[pytest]
pythonpath = .
testpaths = tests
timeout = 60
markers =
    live: uses the real watchdog Observer on a temporary directory

```

```
# Sample data

No sample files are bundled on purpose: the demo builds a fresh synthetic tree each run.

    python scripts/make_sample.py samples/demo_tree --files 120 --seed 7

generates 120 harmless pseudo-random text files (config-, web- and script-like names) under
`samples/demo_tree/`. Nothing in this project scans real system directories.

```

```
#!/usr/bin/env python3
"""End-to-end demonstration on GENERATED files only (no real system scans).

Writes into outputs/: sample_output.txt, demo_events.csv, demo_baseline.csv,
demo_status.json, demo_metrics.json, charts/*.png
Run:  python scripts/demo.py
"""
from __future__ import annotations

import json
import os
import shutil
import sqlite3
import statistics
import subprocess
import sys
import tempfile
import time

HERE = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
sys.path.insert(0, HERE)
sys.path.insert(0, os.path.join(HERE, "scripts"))

from make_sample import generate_bulk, generate_tree  # noqa: E402
from src.fim import FIM  # noqa: E402
from src.report import build_report, export  # noqa: E402
from src.watcher import Watcher  # noqa: E402

OUT = os.path.join(HERE, "outputs")
TMP = tempfile.mkdtemp(prefix="fim-demo-")
ROOT = os.path.join(TMP, "monitored")
DB = os.path.join(TMP, "state", "fim.db")
KEY = os.path.join(TMP, "secrets", "fim.key")
log_lines: list[str] = []


def shown(text: str) -> str:
    return text.replace(TMP, "/tmp/fim-demo")


def cli(*args: str, note: str = "") -> subprocess.CompletedProcess:
    cmd = [sys.executable, "cli.py", "--db", DB, "--key", KEY, *args]
    r = subprocess.run(cmd, capture_output=True, text=True, timeout=120, cwd=HERE)
    if note:
        log_lines.append(f"\n# {note}")
    log_lines.append("$ python cli.py --db /tmp/fim-demo/state/fim.db --key /tmp/fim-demo/secrets/fim.key "
                     + shown(" ".join(args)))
    log_lines.append(shown((r.stdout + r.stderr).rstrip()))
    log_lines.append(f"[exit code {r.returncode}]")
    return r


def live_phase(metrics: dict) -> list[dict]:
    """Scripted changes against the real watchdog Observer; measures latency."""
    fim = FIM(DB, KEY)
    detected: dict[tuple[str, str], float] = {}
    w = Watcher(fim, debounce=0.3, on_event=lambda e: detected.setdefault((e["event_type"], e["path"]), time.monotonic()))
    w.start()
    actions: list[tuple[str, str, float]] = []      # (expected kind, path, t_action)
    outside = os.path.join(TMP, "outside")
    os.makedirs(outside, exist_ok=True)

    def act(kind, rel, fn):
```

```
        t = time.monotonic()
        fn(os.path.join(ROOT, rel))
        actions.append((kind, rel, t))
        time.sleep(0.05)

    for i in range(10):
        act("created", f"var/config/new_{i}.conf", lambda p: open(p, "w").write("new=1\n"))
    for rel in sorted(fim.store.load_baseline())[:10]:
        act("modified", rel, lambda p: open(p, "a").write("# tampered line\n"))
    for rel in sorted(fim.store.load_baseline())[10:15]:
        act("deleted", rel, os.remove)
    for rel in [r for r in sorted(fim.store.load_baseline())[25:] if r.endswith(".sh")][:3]:
        act("modified", rel, lambda p: os.chmod(p, 0o777))
    # burst: 50 writes to one file within ~0.25 s -> expect ONE alert
    burst = sorted(fim.store.load_baseline())[20]
    raw_before = w.raw_events
    t_burst = time.monotonic()
    for i in range(50):
        with open(os.path.join(ROOT, burst), "w") as fh:
            fh.write(f"burst {i}\n")
        time.sleep(0.005)
    actions.append(("modified", burst, t_burst))
    # symlinks pointing outside the tree: must NOT be followed
    os.symlink(outside, os.path.join(ROOT, "srv", "link_to_outside"))
    with open(os.path.join(outside, "secret.txt"), "w") as fh:
        fh.write("outside file\n")
    time.sleep(1.5)
    burst_raw = w.raw_events - raw_before
    w.stop()

    rows = []
    for kind, rel, t in actions:
        td = detected.get((kind, rel))
        rows.append({"kind": kind, "path": rel, "detected": td is not None,
                     "latency_ms": round((td - t) * 1000, 1) if td else None})
    lat = [r["latency_ms"] for r in rows if r["latency_ms"] is not None]
    burst_alerts = sum(1 for e in w.events if e["path"] == burst)
    metrics.update({
        "live_actions": len(rows),
        "live_detected": sum(r["detected"] for r in rows),
        "live_detection_rate_pct": round(100 * sum(r["detected"] for r in rows) / len(rows), 1),
        "live_latency_ms_median": round(statistics.median(lat), 1),
        "live_latency_ms_p95": round(sorted(lat)[int(0.95 * (len(lat) - 1))], 1),
        "live_latency_ms_max": round(max(lat), 1),
        "debounce_s": 0.3,
        "burst_writes": 50,
        "burst_raw_fs_events": burst_raw,
        "burst_alerts": burst_alerts,
        "live_symlink_alerts": sum(1 for e in w.events if "link_to_outside" in (e["path"] or "")),
        "live_symlinks_skipped": w.stats.skipped_symlinks,
        "live_total_raw_events": w.raw_events,
        "live_total_alerts": len(w.events),
    })
    fim.close()
    return rows


def hash_benchmark(metrics: dict) -> None:
    broot = os.path.join(TMP, "bench")
    total = generate_bulk(broot, n_files=400, size=256 * 1024)   # 100 MiB
    f = FIM(os.path.join(TMP, "bench.db"), os.path.join(TMP, "secrets", "bench.key"), root=broot)
    t0 = time.perf_counter()
    res = f.init()
    dt = time.perf_counter() - t0
    t1 = time.perf_counter()
```

```
    scan = f.scan(record=False)
    dt_scan = time.perf_counter() - t1
    metrics.update({"bench_files": res.stats.files, "bench_mib": round(total / 2**20, 1),
                    "bench_init_s": round(dt, 3), "bench_scan_s": round(dt_scan, 3),
                    "bench_throughput_mib_s": round(total / 2**20 / dt_scan, 1),  # warm page cach
e
                    "bench_scan_clean": scan.clean})
    f.close()


def charts(rows: list[dict], counts: dict, metrics: dict) -> None:
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    os.makedirs(os.path.join(OUT, "charts"), exist_ok=True)

    fig, ax = plt.subplots(figsize=(7, 4))
    kinds = ["created", "modified", "deleted"]
    data = [[r["latency_ms"] for r in rows if r["kind"] == k and r["latency_ms"] is not None] for
k in kinds]
    ax.boxplot(data)
    ax.set_xticks([1, 2, 3], kinds)
    for i, d in enumerate(data, 1):
        ax.scatter([i] * len(d), d, alpha=0.6, s=18)
    ax.axhline(metrics["debounce_s"] * 1000, ls="--", color="grey", lw=1, label="debounce window (
300 ms)")
    mx = max((r for r in rows if r["latency_ms"] is not None), key=lambda r: r["latency_ms"])
    ax.annotate("50-write burst (timed from\nits first write)", xy=(kinds.index(mx["kind"]) + 1, m
x["latency_ms"]),
                xytext=(1.6, mx["latency_ms"] * 0.9), arrowprops={"arrowstyle": "->"}, fontsize=8)
    ax.set_ylabel("detection latency (ms)")
    ax.set_title(f"Live watchdog detection latency ({metrics['live_detected']}/{metrics['live_acti
ons']} changes detected)")
    ax.legend()
    fig.tight_layout()
    fig.savefig(os.path.join(OUT, "charts", "detection_latency.png"), dpi=120)
    plt.close(fig)

    fig, ax = plt.subplots(figsize=(7, 4))
    ks = sorted(counts)
    ax.bar(ks, [counts[k] for k in ks], color=["#c0392b" if k == "tamper" else "#2c7fb8" for k in
ks])
    for i, k in enumerate(ks):
        ax.text(i, counts[k], str(counts[k]), ha="center", va="bottom")
    ax.set_title("Audit events recorded during the demo (HMAC-chained)")
    ax.set_ylabel("events")
    fig.tight_layout()
    fig.savefig(os.path.join(OUT, "charts", "events_by_type.png"), dpi=120)
    plt.close(fig)

    fig, ax = plt.subplots(figsize=(6, 3.5))
    ax.bar(["raw filesystem events", "alerts after debounce"],
           [metrics["burst_raw_fs_events"], metrics["burst_alerts"]], color=["#95a5a6", "#27ae60"]
)
    ax.set_title(f"Debouncing a burst of {metrics['burst_writes']} writes to one file")
    for i, v in enumerate([metrics["burst_raw_fs_events"], metrics["burst_alerts"]]):
        ax.text(i, v, str(v), ha="center", va="bottom")
    fig.tight_layout()
    fig.savefig(os.path.join(OUT, "charts", "debounce_burst.png"), dpi=120)
    plt.close(fig)


def main() -> None:
    os.makedirs(OUT, exist_ok=True)
    metrics: dict = {}
```

```
    n_bytes = generate_tree(ROOT, n_files=120, seed=7)
    metrics.update({"sample_files": 120, "sample_bytes": n_bytes})
    log_lines.append(f"# Demo on a GENERATED tree: 120 synthetic files ({n_bytes} bytes), seed 7")
    cli("init", ROOT, note="1. Create the signed baseline (key created with mode 0600 outside the tree)")
    log_lines.append(f"$ stat -c '%a %n' /tmp/fim-demo/secrets/fim.key\n{oct(os.stat(KEY).st_mode & 0o777)[2:]} /tmp/fim-demo/secrets/fim.key")
    cli("scan", note="2. Scan right away: nothing changed")

    # 3. Live CLI watch for the sample output (a bounded subprocess)
    log_lines.append("\n# 3. Live monitoring with watchdog (bounded to 4 s; changes made from another shell)")
    wlog = os.path.join(TMP, "watch.log")
    with open(wlog, "w") as fh:
        proc = subprocess.Popen([sys.executable, "cli.py", "--db", DB, "--key", KEY, "watch",
                                 "--duration", "4", "--debounce", "0.3"], stdout=fh, stderr=subprocess.STDOUT, cwd=HERE)
        time.sleep(1.5)
        open(os.path.join(ROOT, "etc", "app", "injected.conf"), "w").write("listen=0.0.0.0\n")
        first = sorted(FIM(DB, KEY).store.load_baseline())[0]
        open(os.path.join(ROOT, first), "a").write("# edited\n")
        try:
            proc.wait(timeout=30)
        except subprocess.TimeoutExpired:
            proc.kill()
    log_lines.append("$ python cli.py --db ... --key ... watch --duration 4 --debounce 0.3")
    log_lines.append(shown(open(wlog).read().rstrip()))
    # undo so the scripted live phase starts from the baseline
    os.remove(os.path.join(ROOT, "etc", "app", "injected.conf"))

    cli("rebaseline", "--apply", note="Accept the CLI-demo changes explicitly before the measured run")
    rows = live_phase(metrics)
    log_lines.append(f"\n# 4. Measured live run: {metrics['live_detected']}/{metrics['live_actions']} scripted changes "
                     f"detected, median latency {metrics['live_latency_ms_median']} ms; burst of 50 writes -> "
                     f"{metrics['burst_raw_fs_events']} raw events -> {metrics['burst_alerts']} alert(s); "
                     f"symlink alerts: {metrics['live_symlink_alerts']}")
    cli("rebaseline", note="5. Re-baseline is a DRY RUN by default")
    # trim the long dry-run list in the sample output
    out = log_lines[-2].splitlines()
    if len(out) > 14:
        log_lines[-2] = "\n".join(out[:8] + [f"  ... ({len(out) - 10} more lines)"] + out[-2:])
    cli("rebaseline", "--apply", note="   ... and only re-signs with --apply")
    out = log_lines[-2].splitlines()
    if len(out) > 14:
        log_lines[-2] = "\n".join(out[:4] + [f"  ... ({len(out) - 6} more lines)"] + out[-2:])

    # 6. Offline changes while NO watcher runs -> caught by scan
    base = sorted(FIM(DB, KEY).store.load_baseline())
    for rel in base[30:33]:
        open(os.path.join(ROOT, rel), "a").write("offline edit\n")
    for rel in base[40:42]:
        os.remove(os.path.join(ROOT, rel))
    for i in range(2):
        open(os.path.join(ROOT, "opt", "tool", "bin", f"dropped_{i}.sh"), "w").write("#!/bin/sh\n")
    r = cli("scan", note="6. Changes made while the watcher was NOT running are caught by a scan")
    metrics["offline_changes_made"] = 7
    metrics["offline_changes_detected"] = sum(1 for l in r.stdout.splitlines() if l.startswith("  "))

    # 7. Tamper: attacker edits a file AND rewrites its hash in the DB
```

```
    victim = base[30]
    import hashlib
    data = open(os.path.join(ROOT, victim), "rb").read()
    con = sqlite3.connect(DB)
    con.execute("UPDATE baseline SET sha256=?, size=? WHERE path=?", (hashlib.sha256(data).hexdigest(), len(data), victim))
    con.commit(); con.close()
    log_lines.append(f"\n# 7. Attack: rewrite the stored hash of {victim} in SQLite to hide the edit")
    r = cli("scan", note="   scan verifies the HMAC first")
    metrics["tamper_scan_exit_code"] = r.returncode
    cli("verify")
    cli("report", "--last", "6", note="8. Report")

    fim = FIM(DB, KEY)
    rep = build_report(fim, last=50)
    export(fim, os.path.join(OUT, "demo_events.csv"), force=True)
    export(fim, os.path.join(OUT, "demo_baseline.csv"), what="baseline", force=True)
    counts = fim.store.event_counts()
    fim.close()
    json.dump({k: v for k, v in rep.items() if k != "recent_events"} | {"root": "/tmp/fim-demo/monitored"},
              open(os.path.join(OUT, "demo_status.json"), "w"), indent=2)

    hash_benchmark(metrics)
    metrics["event_counts"] = counts
    charts(rows, counts, metrics)
    json.dump({"metrics": metrics, "live_actions": rows}, open(os.path.join(OUT, "demo_metrics.json"), "w"), indent=2)
    log_lines.append(f"\n# 9. Hashing benchmark: {metrics['bench_files']} files / {metrics['bench_mib']} MiB scanned in "
                     f"{metrics['bench_scan_s']} s = {metrics['bench_throughput_mib_s']} MiB/s (files in the OS page cache; cold-disk speed will be lower)")
    with open(os.path.join(OUT, "sample_output.txt"), "w") as fh:
        fh.write("\n".join(log_lines) + "\n")
    print(json.dumps(metrics, indent=2))
    shutil.rmtree(TMP, ignore_errors=True)


if __name__ == "__main__":
    main()

```

02-file-integrity-monitor/scripts/make_sample.py

```
#!/usr/bin/env python3
"""Generate a synthetic, harmless directory tree to monitor (no real system files).

    python scripts/make_sample.py samples/demo_tree --files 120 --seed 7
"""
from __future__ import annotations

import argparse
import os
import random

DIRS = ["etc/app", "etc/nginx", "srv/www/static", "srv/www/templates", "opt/tool/bin", "var/config"]
EXT = [".conf", ".ini", ".html", ".css", ".js", ".sh", ".json", ".txt"]


def generate_tree(root: str, n_files: int = 120, seed: int = 7, size_range=(200, 8000)) -> int:
    """Create ``n_files`` deterministic pseudo-random text files under ``root``."""
    rng = random.Random(seed)
    total = 0
    for i in range(n_files):
        d = os.path.join(root, rng.choice(DIRS))
        os.makedirs(d, exist_ok=True)
        name = f"file_{i:04d}{rng.choice(EXT)}"
        size = rng.randint(*size_range)
        words = [rng.choice(["alpha", "beta", "gamma", "delta", "config", "value", "enabled",
                             "port", "host", "timeout", "user", "path"]) for _ in range(size // 6)]
        body = f"# synthetic sample file {i} (generated, seed={seed})\n" + " ".join(words)[:size] + "\n"
        path = os.path.join(d, name)
        with open(path, "w") as fh:
            fh.write(body)
        if name.endswith(".sh"):
            os.chmod(path, 0o755)
        total += len(body)
    return total


def generate_bulk(root: str, n_files: int, size: int, seed: int = 1) -> int:
    """Binary files for the hashing throughput benchmark."""
    rng = random.Random(seed)
    os.makedirs(root, exist_ok=True)
    for i in range(n_files):
        with open(os.path.join(root, f"blob_{i:04d}.bin"), "wb") as fh:
            fh.write(rng.randbytes(size))
    return n_files * size


if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("root")
    ap.add_argument("--files", type=int, default=120)
    ap.add_argument("--seed", type=int, default=7)
    a = ap.parse_args()
    n = generate_tree(a.root, a.files, a.seed)
    print(f"generated {a.files} files ({n} bytes) under {a.root}")

```